Java Coding Interview Revision Book

60 worked problems for a CodeSignal-style screen

Final-year engineering level

Every problem has a complete Java solution, its complexity,
and the mistake that most often costs the points.

Companion to `ASSESSMENT.md` — work the book first, sit the assessment second.

How to use this book

The screen you are preparing for is reported to be four problems in seventy minutes on CodeSignal, with tasks in the range of *array manipulation* and *data processing: calculating averages, filtering data*. That is the bar. It is not a competitive-programming bar, and almost nothing in rounds like this needs dynamic programming or graph theory.

What it does need is that you write correct Java *quickly*, without stopping to remember how a `HashMap` iterates or what `split(",")` does to trailing empty fields. Speed on easy and medium problems is worth far more than the ability to crack one hard one.

A working method.

1. Read the whole statement before typing. Note every edge case it names: `null`, empty, ties, bounds. They are in the statement because they are in the tests.
2. Say the approach out loud in one sentence. If you cannot, you do not have one yet.
3. Write the happy path. Then go back and add the edge cases you noted.
4. Re-read the statement against your code before moving on. The most expensive bug in a timed test is the one you never look for.

How to drill. Give yourself the time budget printed on each difficulty: Easy 8 minutes, Medium 15, Hard 25. Write the solution without looking. Then read the **Trap** note — it is the thing that actually breaks submissions, and it is worth more than the solution itself.

Chapters 1 to 8 are the exam. Chapter 9 is deliberately above the bar. Work it only when the rest is comfortable; it is there so that a hard problem on the day does not surprise you.

Contents

How to use this book **1**

1 The Java toolkit **4**
Choosing the container . . . 4
Idioms worth memorising . . . 4
Traps that cost real points . . . 5
Complexity you should recognise instantly . . . 5
How to read a diagram in this book . . . 6

2 Strings **7**
P1. Reverse the words in a sentence . . . 7
P2. Valid palindrome, ignoring punctuation . . . 7
P3. First non-repeating character . . . 8
P4. Run-length encoding . . . 9
P5. Anagram check . . . 9
P6. Longest common prefix . . . 10
P7. Compare two version strings . . . 11
P8. Longest substring without repeating characters . . . 11

3 Arrays and two pointers **13**
P9. Move zeros to the end . . . 13
P10. Remove duplicates from a sorted array . . . 13
P11. Two-sum in a sorted array . . . 14
P12. Maximum subarray sum . . . 15
P13. Rotate an array by k . . . 15
P14. Merge two sorted arrays . . . 16
P15. Product of the array except self . . . 17
P16. Container with most water . . . 17

4 Hashing and counting **19**
P17. Count pairs summing to a target . . . 19
P18. Longest subarray with equal counts . . . 19
P19. Group anagrams . . . 20
P20. Top k frequent elements . . . 21
P21. Subarray sum equals k . . . 21
P22. First duplicate . . . 22
P23. Intersection of two arrays . . . 23
P24. Isomorphic strings . . . 23

5 Sorting, intervals and comparators **25**
P25. Sort by frequency, then by value . . . 25
P26. Merge overlapping intervals . . . 25
P27. Can one person attend all meetings? . . . 26
P28. Minimum number of meeting rooms . . . 27
P29. Insert an interval . . . 28
P30. Sort employees by salary then name . . . 28
P31. Kth largest element . . . 29

6 Matrices **30**
P32. Rotate a square matrix 90 degrees clockwise . . . 30
P33. Spiral traversal . . . 30
P34. Count islands . . . 31
P35. Set matrix zeroes . . . 32
P36. Diagonal sums . . . 33
P37. Search a row-and-column sorted matrix . . . 34

7 Recursion, BFS and DFS **35**
P38. Fibonacci with memoisation . . . 35
P39. Maximum depth of a binary tree . . . 35
P40. Level-order traversal . . . 36
P41. Flood fill . . . 37
P42. All permutations of a string . . . 38
P43. All subsets . . . 38
P44. Shortest path in a grid with obstacles . . . 39

8 Stacks and queues **41**
P45. Valid parentheses . . . 41
P46. Evaluate reverse Polish notation . . . 41
P47. Next greater element . . . 42
P48. Min stack . . . 43
P49. Simplify a Unix path . . . 44

9 Data processing **45**
P50. Average salary by department . . . 45
P51. Filter and total transactions . . . 46
P52. Validate records and report the errors . . . 47
P53. Join two datasets on a key . . . 48
P54. Bucket events by hour and find the peak . . . 48
P55. Running balance per account . . . 49

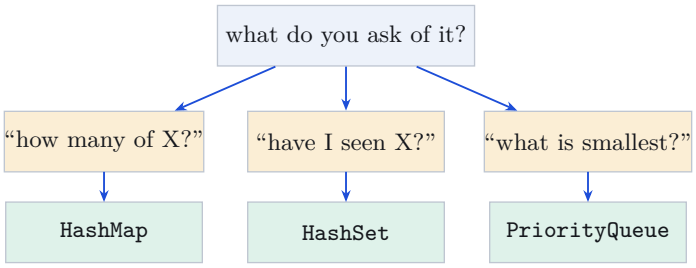
10 Stretch **51**
P56. Minimum travel time with k free roads . . . 51
P57. Expiring files . . . 52
P58. LRU cache . . . 53
P59. Coin change . . . 54
P60. Longest increasing subsequence . . . 54

1. The Java toolkit

The things you must not have to think about on the day. Read this once; come back whenever a drill makes you hesitate.

Choosing the container

Almost every problem in this book is solved by picking the right container and then writing five lines. The question to ask is not "which collection do I know" but "what question will I ask of this data, over and over".



You need	Use	Why / cost
Count things by key	`HashMap<K,Integer>`	`merge(k,1,Integer::sum)`, O(1)
Group things by key	`HashMap<K,List<V»`	`computeIfAbsent(k, x -> new ArrayList<>())`
Seen-before / dedupe	`HashSet<T>`	`add` returns `false` if present
Keys in sorted order	`TreeMap<K,V>`	O(log n), gives `firstKey`, `floorKey`
Insertion order kept	`LinkedHashMap`	iteration order = insertion order
Growable array	`ArrayList<T>`	O(1) amortised add, O(1) get
Stack or queue	`ArrayDeque<T>`	`push/pop`, `offer/poll`; never `Stack`
Top-k / smallest-first	`PriorityQueue<T>`	O(log n) per op, min-heap by default

`ArrayDeque` is the right answer for both stacks and queues. `java.util.Stack` is a synchronised relic and `LinkedList` costs you a pointer chase per element.

Idioms worth memorising

```
// count occurrences
Map<Character,Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) freq.merge(c, 1, Integer::sum);

// group into lists
Map<String,List<String>> byKey = new HashMap<>();
byKey.computeIfAbsent(key, k -> new ArrayList<>()).add(value);

// accumulate a total and a count together
map.computeIfAbsent(dept, k -> new long[2])[0] += salary;   // [0]=total
map.get(dept)[1]++;                                          // [1]=count

// sort with a tie-break: primary descending, secondary ascending
list.sort((a, b) -> a.score != b.score
        ? Integer.compare(b.score, a.score)
        : a.name.compareTo(b.name));

// iterate a map
for (Map.Entry<String,Integer> e : map.entrySet())
    use(e.getKey(), e.getValue());
```

```java
// array <-> list
int[] arr = list.stream().mapToInt(Integer::intValue).toArray();
String[] out = list.toArray(new String[0]);

// 2D array, all cells set
char[][] g = new char[rows][cols];
for (char[] row : g) Arrays.fill(row, '.');

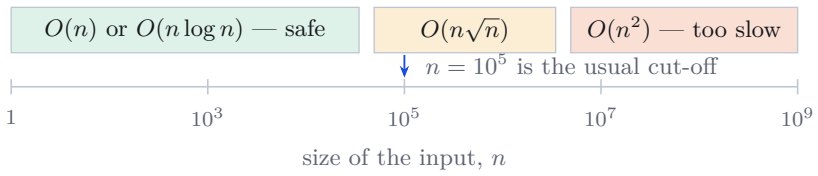
// the four neighbours (eight: nest -1..1 and skip 0,0)
int[][] DIRS = {{-1,0},{1,0},{0,-1},{0,1}};
```

Traps that cost real points

- **`split(",")` drops trailing empties.** `"a,b,"` splits into 2 parts, not 3. Use `split(",", -1)` whenever the field count is part of your validation.
- **`%` keeps the sign of the dividend.** `-3 % 2 == -1`, so `n % 2 == 1` is false for negative odd numbers. Write `n % 2 != 0`.
- **Integer division truncates toward zero.** `7/2 == 3` and `-7/2 == -3`. That is not floor for negatives.
- **`int` overflow is silent.** Sums of up to 2×10^9 values overflow. Accumulate in `long`.
- **`==` on boxed types compares references.** Use `.equals`, or unbox to `int` first. `Integer` caches only -128 to 127, which is why small cases seem to work.
- **`String +=` in a loop is $O(n^2)$.** Use `StringBuilder`.
- **`ConcurrentModificationException`** when you add to or remove from a collection while a for-each is running. Collect the keys first, then mutate; or use `Iterator.remove()`.
- **A recursive flood fill overflows the stack** at a few tens of thousands of cells. Use an explicit `ArrayDeque`.
- **Returning `null` where an empty array was asked for.** Read the return contract twice.

Complexity you should recognise instantly

The constraint line in a statement is the examiner telling you which complexity they expect. Read it before you choose an approach, not after your first submission times out.



Shape	Cost	Safe up to
Single pass	$O(n)$	10^8
Sort	$O(n \log n)$	10^7
Nested loop over the input	$O(n^2)$	$\sim 10^4$
Pass per element, each a scan	$O(n^2)$	the same, and this is the one you write by accident
Flood fill / BFS on a grid	$O(\text{rows} \times \text{cols})$	10^7 cells

How to read a diagram in this book

Every problem has one. The conventions are constant throughout:



2. Strings

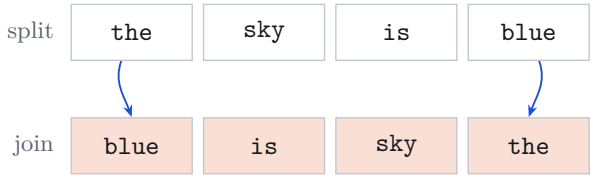
Eight problems. Strings open most screens because they are quick to state and full of edge cases.

P1. Reverse the words in a sentence

Easy

Given a sentence, return it with the words in reverse order and exactly one space between them. Leading, trailing and repeated spaces disappear.

Idea. Split on runs of whitespace, walk the parts backwards, join with a single space.



How it works. The whole problem is choosing the right separator. A sentence like `" the sky is blue "` has a leading space, a double space in the middle and a trailing space. `trim()` removes the two ends; the regular expression `\s+` means "one or more whitespace characters", so a run of any length counts as a single separator. With those two steps the array holds exactly the words, and reversing it is a backwards loop.

Solution

```
public static String reverseWords(String s) {
    if (s == null) return "";
    String[] parts = s.trim().split("\\s+");
    StringBuilder out = new StringBuilder();
    for (int i = parts.length - 1; i >= 0; i--) {
        if (parts[i].isEmpty()) continue;
        if (out.length() > 0) out.append(' ');
        out.append(parts[i]);
    }
    return out.toString();
}
```

The `out.length() > 0` test is how you get spaces *between* words without a trailing one: append the separator before each word except the first.

Cost. $O(n)$ time, $O(n)$ space.

Trap. Splitting on `" "` instead of `"\\s+"` leaves empty strings in the array when spaces repeat. Forgetting `trim()` does the same at the ends.

P2. Valid palindrome, ignoring punctuation

Easy

Return `true` if the string reads the same both ways when you consider only letters and digits, and ignore case.

Idea. Two pointers walking inwards, each skipping anything that is not alphanumeric.



each pointer skips over what is not a letter or digit

How it works. Rather than building a cleaned copy of the string (which costs a second array), the two pointers do the cleaning as they move. The outer `while` advances them towards each other;

the two inner `while` loops consume any run of punctuation before a comparison is made. Only when both pointers are parked on real characters does the comparison happen, and then both step inward together.

The subtle part is that the inner loops need their own `i < j` guard. Without it, a string of pure punctuation would run `i` straight off the end of the string and throw.

Solution

```
public static boolean isPalindrome(String s) {
    if (s == null) return true;
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i)) !=
            Character.toLowerCase(s.charAt(j))) return false;
        i++; j--;
    }
    return true;
}
```

Cost. $O(n)$ time, $O(1)$ space — nothing is allocated.

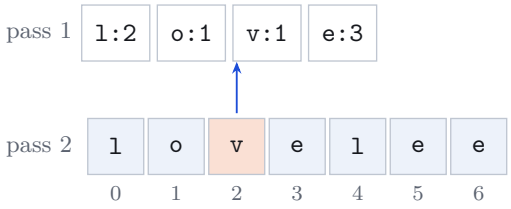
Trap. Dropping the `i < j` guard from the skip loops. Also, comparing without `toLowerCase` on both sides: `'A'` and `'a'` are different `char` values.

P3. First non-repeating character

Easy

Return the index of the first character that appears exactly once, or -1.

Idea. Count every character in one pass, then scan the *string* again in order and return the first whose count is 1.



index 2 is the first character whose count is 1

How it works. Two passes, and the order of the second one is the whole point. After the counting pass you know how often every character occurs, but a `HashMap` has no order, so you cannot ask it "which of these came first". The string itself still knows. Re-scanning it in index order and stopping at the first count of 1 gives the answer directly.

For a fixed alphabet an `int[128]` beats a map: no boxing, no hashing, and the array is cheap to allocate.

Solution

```
public static int firstUnique(String s) {
    if (s == null || s.isEmpty()) return -1;
    int[] freq = new int[128];                  // ASCII
    for (int i = 0; i < s.length(); i++) freq[s.charAt(i)]++;
    for (int i = 0; i < s.length(); i++)
        if (freq[s.charAt(i)] == 1) return i;
    return -1;
}
```

Cost. $O(n)$ time, $O(1)$ space — the table is a fixed 128 entries.

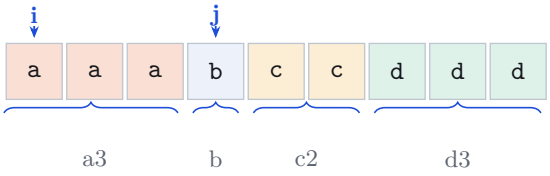
Trap. Scanning the map rather than the string on the second pass. If the input can be any Unicode character, `int[128]` overflows its bounds — use a `LinkedHashMap`, which does preserve insertion order.

P4. Run-length encoding

Easy

Replace each maximal run of identical characters with the character followed by the run length, writing a run of one as just the character.

Idea. One pointer at the start of the run, a second scanning forward while the character is unchanged.



How it works. `i` marks where the current run starts. `j` runs ahead for as long as the character matches, so when it stops, `j - i` is exactly the run length and `j` is the start of the next run. Setting `i = j` at the end of each iteration means every character is examined once by `j` and never revisited — the two pointers together make one pass over the string, not a pass per run.

Appending the length only when `j - i > 1` is what produces `b` rather than `b1`.

Solution

```
public static String compress(String s) {
    if (s == null || s.isEmpty()) return "";
    StringBuilder out = new StringBuilder();
    int i = 0;
    while (i < s.length()) {
        char ch = s.charAt(i);
        int j = i;
        while (j < s.length() && s.charAt(j) == ch) j++;
        out.append(ch);
        if (j - i > 1) out.append(j - i);
        i = j;
    }
    return out.toString();
}
```

Cost. $O(n)$ time, $O(n)$ space for the output.

Trap. Run lengths can have more than one digit — `"z12"` is a valid output, so anything that appends a single `char` for the count breaks past nine. Building the result with `String +=` turns this into $O(n^2)$.

Also appears as: Task 1 of the assessment.

P5. Anagram check

Easy

Return `true` if two strings contain the same letters with the same multiplicities.

Idea. Count up for the first string and down for the second in the same table. If nothing is left over, they match.

	a	d	e	g
"egg"	0	0	+1	+2
"ged"	0	−1	−1	−1
total	0	−1	0	+1

two non-zero entries, so not an anagram

How it works. An anagram is a statement about counts and nothing else, so you never need to sort. Walking both strings in the same loop is only possible because a length mismatch is rejected first — that early return is doing real work, not just guarding.

The array index `a.charAt(i) - 'a'` maps `'a'` to 0, `'b'` to 1 and so on. It is fast and it is also the assumption that breaks if the input is not pure lowercase.

Solution

```
public static boolean isAnagram(String a, String b) {
    if (a == null || b == null || a.length() != b.length()) return false;
    int[] freq = new int[26];
    for (int i = 0; i < a.length(); i++) {
        freq[a.charAt(i) - 'a']++;
        freq[b.charAt(i) - 'a']--;
    }
    for (int f : freq) if (f != 0) return false;
    return true;
}
```

Cost. $O(n)$ time, $O(1)$ space.

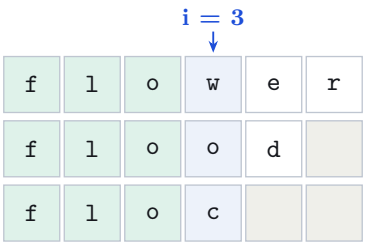
Trap. `- 'a'` assumes lowercase. An uppercase letter gives a negative index and throws; a digit or space lands outside 0–25 too. For arbitrary input use a `HashMap` or a 128-entry table.

P6. Longest common prefix

Easy

Return the longest prefix shared by every string in an array, or `""`.

Idea. Compare column by column rather than string by string: take character i of the first word and check it against character i of all the others.



first mismatch at column 3, so the answer is `"flo"`

How it works. The prefix can never be longer than the shortest word, and it can never survive a disagreement in any column. Scanning vertically means you stop at the first column that fails, and that column index is exactly the length of the answer. The outer loop is bounded by the first word's length, so the inner guard `i >= words[w].length()` is what handles a word that is shorter than the first one.

Solution

```
public static String longestCommonPrefix(String[] words) {
    if (words == null || words.length == 0) return "";
    for (int i = 0; i < words[0].length(); i++) {
        char c = words[0].charAt(i);
```

```
        for (int w = 1; w < words.length; w++) {
            if (words[w] == null) return "";
            if (i >= words[w].length() || words[w].charAt(i) != c)
                return words[0].substring(0, i);
        }
    }
    return words[0];
}
```

Cost. O(total characters) in the worst case, $O(1)$ extra space.

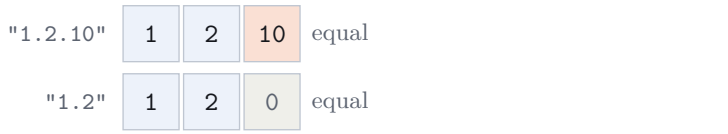
Trap. Omitting the length check throws `StringIndexOutOfBounds` the moment one word is a strict prefix of another, as in `["ab", "a"]`.

P7. Compare two version strings

Medium

Versions are dot-separated numbers. Return -1, 0 or 1. Missing components count as zero, so `"1.0"` equals `"1"`.

Idea. Split both, walk to the longer length, and treat a missing component as zero. Compare numerically, never as text.



the missing third component is read as 0, so `"1.2.10"` wins

How it works. Padding with zeros instead of stopping at the shorter array is what makes `"1.0"` and `"1"` compare equal. The loop runs to `max(x.length, y.length)` and the conditional expression supplies a 0 whenever an index is past the end of one side.

Parsing each component into a number is the other half. As text, `"10"` sorts before `"9"` because `'1' < '9'`; as numbers, it does not.

Solution

```
public static int compareVersions(String a, String b) {
    String[] x = a.split("\\.");
    String[] y = b.split("\\.");
    int n = Math.max(x.length, y.length);
    for (int i = 0; i < n; i++) {
        long left  = i < x.length ? Long.parseLong(x[i]) : 0;
        long right = i < y.length ? Long.parseLong(y[i]) : 0;
        if (left != right) return left < right ? -1 : 1;
    }
    return 0;
}
```

Cost. $O(n)$ time.

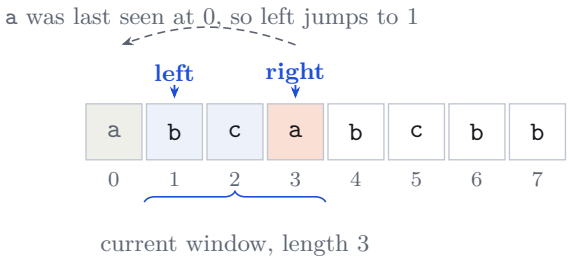
Trap. `split(".")` splits on *every character*, because `.` is a regex wildcard matching anything — you must escape it as `"\\."`. The result is an empty array and a silent wrong answer.

P8. Longest substring without repeating characters

Medium

Return the length of the longest substring in which no character repeats.

Idea. A sliding window. Remember the last index at which each character was seen; when you meet a repeat that lies inside the window, jump the left edge past it.



How it works. The window `[left, right]` is maintained as an invariant: it never contains a repeat. Each step extends `right` by one and then repairs the invariant if the new character broke it.

The repair is the interesting line. If the character was last seen at index `prev`, every window containing both occurrences is invalid, so the left edge must move to `prev + 1`. But only if `prev` is *inside* the current window — hence `prev >= left`. A character seen long ago and already excluded must not drag the left edge backwards; the window only ever grows to the right.

Solution

```java
public static int longestUnique(String s) {
    if (s == null || s.isEmpty()) return 0;
    Map<Character,Integer> lastSeen = new HashMap<>();
    int best = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        Integer prev = lastSeen.get(c);
        if (prev != null && prev >= left) left = prev + 1;
        lastSeen.put(c, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Cost. $O(n)$ time — each index is visited once by `right` and the left edge only moves forward. $O(\min(n, \Sigma))$ space.

Trap. Omitting `prev >= left`. The code still runs and gives the right answer on short examples, then fails on inputs where a character recurs after leaving the window.

3. Arrays and two pointers

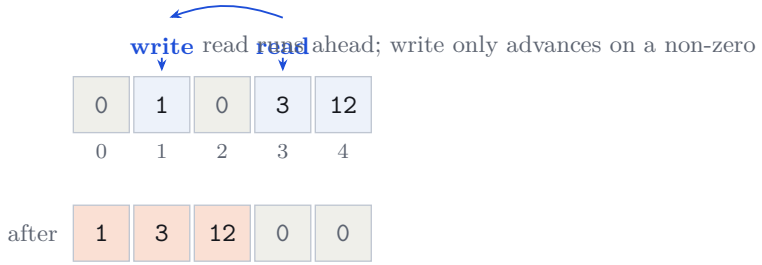
Eight problems. A read index and a write index, or two ends closing in — this one idea turns a great many quadratic solutions linear.

P9. Move zeros to the end

Easy

Move every zero to the end, keeping the order of the non-zero elements, in place.

Idea. A write pointer. Copy every non-zero forward to the write position, then fill the tail with zeros.



How it works. Two indices move at different speeds over the same array. `read` visits every slot; `write` only advances when something is actually kept. Because `write` can never get ahead of `read`, copying `a[read]` into `a[write]` can only ever overwrite a slot that has already been read — which is why this is safe to do in place with no temporary array.

When the first loop ends, `write` is the count of non-zero elements, so everything from there to the end is tail that must be zeroed.

Solution

```
public static void moveZeros(int[] a) {
    if (a == null) return;
    int write = 0;
    for (int read = 0; read < a.length; read++)
        if (a[read] != 0) a[write++] = a[read];
    while (write < a.length) a[write++] = 0;
}
```

Cost. $O(n)$ time, $O(1)$ space.

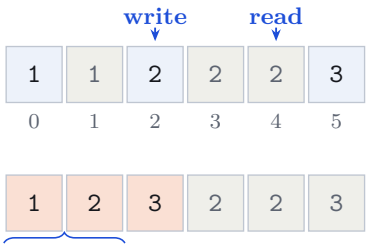
Trap. Swapping with the *end* of the array instead of copying to `write` destroys the relative order of the non-zero elements, which the statement asks you to preserve.

P10. Remove duplicates from a sorted array

Easy

Compact a sorted array in place so each value appears once; return the new length.

Idea. The same write pointer. Because the array is sorted, duplicates are always adjacent, so compare each element with the last one you kept.



returned length 3; the rest is ignored

How it works. Sortedness is doing the heavy lifting: it guarantees that if a value is going to repeat, it repeats immediately. So the test for "have I seen this already" collapses from a set lookup to a single comparison with the previous kept element.

That comparison must be against `a[write - 1]`, not `a[read - 1]`. Once you start overwriting, `a[read - 1]` may be a value that was copied there this pass, so it no longer tells you what the last distinct value was.

Solution

```
public static int dedupeSorted(int[] a) {
    if (a == null || a.length == 0) return 0;
    int write = 1;
    for (int read = 1; read < a.length; read++)
        if (a[read] != a[write - 1]) a[write++] = a[read];
    return write;
}
```

Cost. $O(n)$ time, $O(1)$ space.

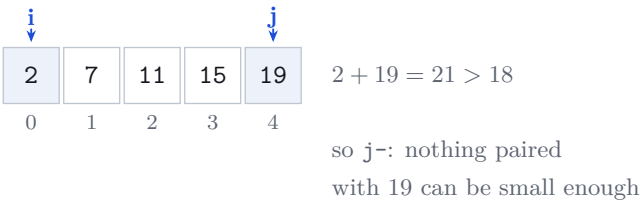
Trap. Comparing against `a[read - 1]`. It looks equivalent and passes on inputs with no more than two consecutive duplicates.

P11. Two-sum in a sorted array

Easy

Return the indices of two values that add to a target, or `null`.

Idea. One pointer at each end. If the sum is too big the right pointer must come in; if it is too small the left must go out. No other move can help.



How it works. The argument for why nothing is missed matters more than the code. Suppose the sum at the current pair is too large. Every pair using the right-hand value and *any* remaining left-hand value is at least this large, because the array is sorted ascending and the left pointer only ever moves right. So that right-hand value cannot be part of any solution, and discarding it loses nothing. The symmetric argument handles the too-small case.

Each step eliminates one element, so the search finishes in at most n steps — the same answer a hash map would give, with no extra memory.

Solution

```
public static int[] twoSumSorted(int[] a, int target) {
    if (a == null) return null;
    int i = 0, j = a.length - 1;
    while (i < j) {
        long sum = (long) a[i] + a[j];
        if (sum == target) return new int[] {i, j};
        if (sum < target) i++; else j--;
    }
    return null;
}
```

On an *unsorted* array this does not apply — use a `HashMap` from value to index instead, also $O(n)$.

Cost. $O(n)$ time, $O(1)$ space.

Trap. `a[i] + a[j]` overflows `int` near the limits. Widen to `long` before adding, as above.

P12. Maximum subarray sum

Medium

Return the largest sum obtainable from a contiguous, non-empty subarray.
`[-2,1,-3,4,-1,2,1,-5,4]` gives 6.

Idea. Kadane. At each element the best subarray ending here is either this element alone, or this element extending the best one ending just before it.



How it works. The second row is the whole algorithm. At each index you only ever ask one question: is it better to start fresh here, or to carry forward what came before? If the running total so far is negative it can only hurt, so you drop it — that is exactly what `Math.max(a[i], endingHere + a[i])` encodes.

Notice that the best overall answer is not necessarily the last value of the running total; it is the maximum ever reached. That is why `best` is tracked separately.

Solution

```
public static int maxSubarraySum(int[] a) {
    if (a == null || a.length == 0) return 0;
    long best = a[0], endingHere = a[0];
    for (int i = 1; i < a.length; i++) {
        endingHere = Math.max(a[i], endingHere + a[i]);
        best = Math.max(best, endingHere);
    }
    return (int) best;
}
```

Cost. $O(n)$ time, $O(1)$ space.

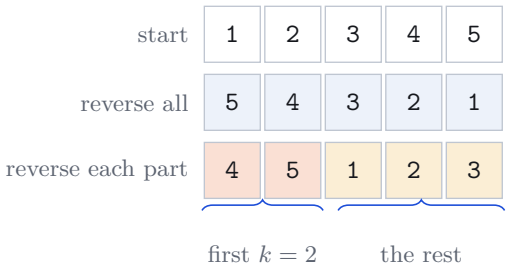
Trap. Initialising `best` to `0` returns 0 on an all-negative array, when the answer is the least negative element. The statement says non-empty, so the empty subarray is not an option.

P13. Rotate an array by k

Medium

Shift every element right by k positions, wrapping around, in place.

Idea. Reverse the whole array, then reverse the first k and the rest separately.



How it works. Rotating right by k means the last k elements move to the front, keeping their order, and the rest follow, keeping theirs. Reversing the whole array brings the last k to the front but in the wrong order, and does the same to the rest. Reversing each of those two blocks individually puts both back in order. Three linear reversals, no second array.

The normalisation `k = ((k % n) + n) % n` handles two cases at once: k larger than the array, and negative k. A single `%` leaves a negative result negative.

Solution

```java
public static void rotate(int[] a, int k) {
    if (a == null || a.length == 0) return;
    k = ((k % a.length) + a.length) % a.length;   // handles negative k
    reverse(a, 0, a.length - 1);
    reverse(a, 0, k - 1);
    reverse(a, k, a.length - 1);
}

private static void reverse(int[] a, int i, int j) {
    while (i < j) {
        int t = a[i]; a[i] = a[j]; a[j] = t;
        i++; j--;
    }
}
```

Cost. $O(n)$ time, $O(1)$ space.

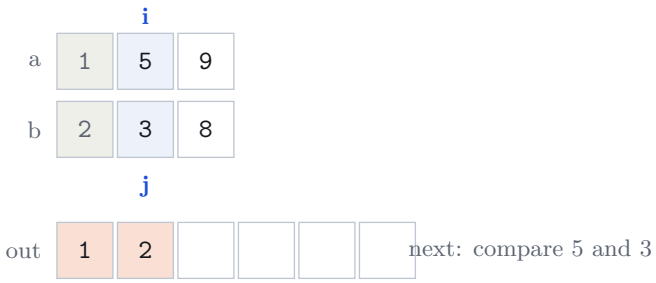
Trap. Not normalising k. With `k = 7` on a length-5 array, `reverse(a, 0, 6)` indexes past the end and throws.

P14. Merge two sorted arrays

Easy

Both inputs are sorted ascending. Return one sorted array with all the elements.

Idea. One index into each; repeatedly take the smaller head, then drain whichever array still has elements.



How it works. At every step the smallest element not yet taken must be at the head of one of the two arrays, because each is sorted. So one comparison is enough to decide what goes next — there is no searching.

The two drain loops after the main one are not optional. The main loop exits as soon as *either* array runs out, and whatever remains in the other is already sorted and larger than everything written so far, so it can be copied straight across.

Using `<=` rather than `<` in the comparison keeps the merge stable: equal elements from `a` come out before those from `b`.

Solution

```java
public static int[] merge(int[] a, int[] b) {
    if (a == null) a = new int[0];
    if (b == null) b = new int[0];
    int[] out = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length)
        out[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < a.length) out[k++] = a[i++];
    while (j < b.length) out[k++] = b[j++];
    return out;
}
```

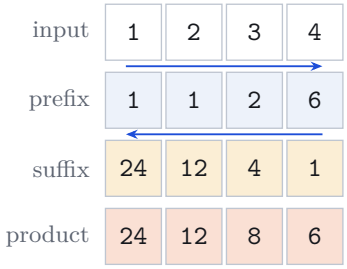
Cost. $O(n + m)$ time and space.

Trap. Forgetting the drain loops, which silently truncates the output.

P15. Product of the array except self

Medium

Position i of the result holds the product of every element except `a[i]`. No division allowed.
Idea. Two sweeps. Left to right, store the product of everything *before* i. Right to left, multiply in the product of everything *after* i, carried in a single variable.



How it works. The product of everything except position i splits cleanly into two halves: what is to its left and what is to its right. Each half is a running product, computable in one sweep.

The implementation is tighter than the diagram suggests. The first loop writes the prefix products directly into the output array; the second loop walks backwards, multiplying each slot by a running suffix carried in one variable. So only the output array is allocated, and the extra space is $O(1)$.

Solution

```java
public static int[] productExceptSelf(int[] a) {
    if (a == null || a.length == 0) return new int[0];
    int n = a.length;
    int[] out = new int[n];
    out[0] = 1;
    for (int i = 1; i < n; i++) out[i] = out[i - 1] * a[i - 1];
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        out[i] *= suffix;
        suffix *= a[i];
    }
    return out;
}
```

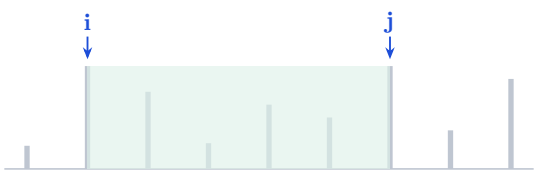
Cost. $O(n)$ time, $O(1)$ extra space beyond the output.

Trap. The division shortcut — total product divided by `a[i]` — breaks on one zero and is undefined on two. That is exactly why the statement forbids it.

P16. Container with most water

Medium

Each element is the height of a vertical line. Pick two so the rectangle between them holds the most water: the shorter height times the distance.
Idea. Start at the widest pair and close in, always moving the *shorter* line.



area $= \min(8, 8) \times 5 = 40$

How it works. The area is limited by the shorter of the two lines, so think about what happens when you move each one inward. Moving the *taller* line loses width and cannot raise the limiting height — the shorter line is still there, still capping the area. So no better rectangle uses that pair of positions, and the move is strictly wasteful.

Moving the shorter line, by contrast, at least gives the height a chance to increase. That asymmetry is what makes it safe to discard one candidate per step and finish in a single pass.

Solution

```java
public static int maxArea(int[] height) {
    if (height == null || height.length < 2) return 0;
    int i = 0, j = height.length - 1, best = 0;
    while (i < j) {
        int h = Math.min(height[i], height[j]);
        best = Math.max(best, h * (j - i));
        if (height[i] < height[j]) i++; else j--;
    }
    return best;
}
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. Moving whichever pointer you feel like, or both at once. The proof that nothing is missed depends on always discarding the shorter line.

4. Hashing and counting

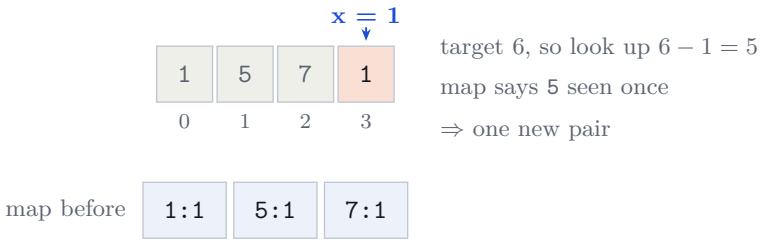
Eight problems. If a question asks how many, have I seen, *or* group by, *the answer starts with a* `HashMap`. *The prefix trick in P18 and P21 is the highest-value pattern in this book.*

P17. Count pairs summing to a target

Easy

Count the pairs of indices $i < j$ with `a[i] + a[j] == target`.

Idea. Walk once. For each element, the number of earlier elements that complete the pair is already in the map. Count first, then insert.



How it works. Every pair has a later element and an earlier one. If you always count from the point of view of the later element, each pair is counted exactly once — no division by two, no double counting.

That is why the order of the two statements matters. Looking up the complement *before* inserting the current value means the map holds only elements strictly to the left. Insert first and an element can pair with itself: a single `3` with target 6 would report a pair that does not exist.

Solution

```
public static long countPairs(int[] a, int target) {
    if (a == null) return 0;
    Map<Integer,Integer> seen = new HashMap<>();
    long pairs = 0;
    for (int x : a) {
        pairs += seen.getOrDefault(target - x, 0);
        seen.merge(x, 1, Integer::sum);
    }
    return pairs;
}
```

Cost. $O(n)$ time, $O(n)$ space.

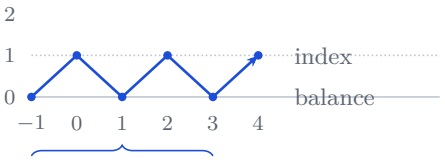
Trap. Returning `int`. With 200 000 identical elements the count is about 2×10^{10}, which overflows.

P18. Longest subarray with equal counts

Medium

Return the length of the longest contiguous subarray holding as many even numbers as odd ones.

Idea. Turn it into a running balance: $+1$ for odd, -1 for even. A subarray is balanced exactly when the balance at its two ends is equal.



same balance at both ends ⇒ length 4

How it works. Write B_i for the balance after the first i elements. The subarray from $j+1$ to i has as many odds as evens exactly when $B_i = B_j$, because the difference $B_i - B_j$ counts (odds − evens) inside that window.

So the problem becomes: for each balance value, how far apart are its two furthest occurrences? Since the balance is scanned left to right, you only ever need the *earliest* index at which each value appeared — any later repeat measured against it gives the longest span. Hence `putIfAbsent` semantics: write the index only when the value is new.

The seed `firstIndex.put(0, -1)` represents the empty prefix. Without it, a subarray that is balanced from index 0 has no earlier index to measure against and is missed entirely.

Solution

```java
public static int longestBalanced(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    Map<Integer,Integer> firstIndex = new HashMap<>();
    firstIndex.put(0, -1);                  // the empty prefix
    int balance = 0, best = 0;
    for (int i = 0; i < nums.length; i++) {
        balance += (nums[i] % 2 != 0) ? 1 : -1;
        Integer seen = firstIndex.get(balance);
        if (seen == null) firstIndex.put(balance, i);
        else best = Math.max(best, i - seen);
    }
    return best;
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. Three in one problem. Overwriting the stored index on a repeat measures the shortest span. Omitting the `0 -> -1` seed loses answers starting at index 0. And `nums[i] % 2 == 1` is false for -3.

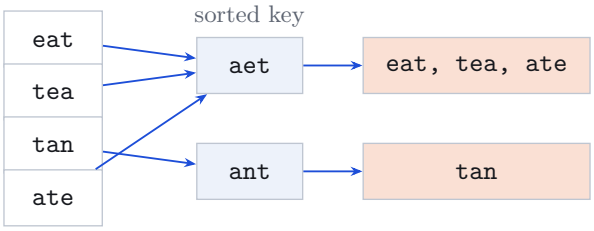
Also appears as: Task 4 of the assessment; and with 0 and 1 in place of even and odd, the classic "contiguous array" question.

P19. Group anagrams

Medium

Group the words that are anagrams of one another.
`["eat","tea","tan","ate"]` gives `[[eat,tea,ate],[tan]]`.

Idea. Give every word a canonical key that all its anagrams share. Sorting its letters is the simplest such key.



How it works. Two words are anagrams precisely when they have the same multiset of letters. A multiset has no natural identity in Java, so you manufacture one: sort the letters and the result is identical for all members of a group and different for everything else. That string becomes the map key, and `computeIfAbsent` does the grouping in one line.

A 26-slot count rendered as a string is a faster key — it drops the $\log k$ sorting factor — but sorting is shorter to write and almost always fast enough.

Solution

```java
public static List<List<String>> groupAnagrams(String[] words) {
    Map<String,List<String>> groups = new HashMap<>();
    for (String w : words) {
        char[] letters = w.toCharArray();
        Arrays.sort(letters);
        String key = new String(letters);
        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(groups.values());
}
```

Cost. $O(n\,k \log k)$ for n words of length k.

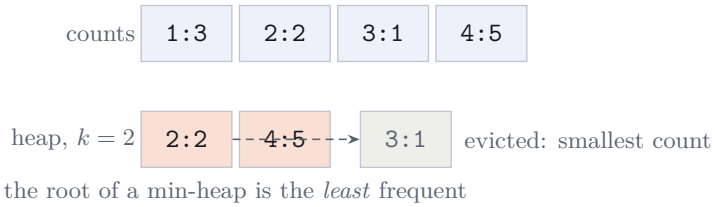
Trap. Using the word's `hashCode` or the sum of its characters as the key. Both collide — `"ad"` and `"bc"` have the same letter sum — and the bug only shows on inputs you did not try.

P20. Top k frequent elements

Medium

Return the k most frequent values, most frequent first.

Idea. Count, then select. A min-heap capped at size k keeps the selection at $O(n \log k)$.



How it works. Counting is the easy half. The selection is where the choice lies: sorting all distinct values is $O(d \log d)$, while a heap capped at k is $O(d \log k)$ and, more importantly, needs only k entries in memory — which is what makes it work on a stream.

The counter-intuitive part is that you keep the top k with a *min*-heap. Push everything; whenever the heap exceeds k, discard its root, which is the least frequent thing currently held. What survives is the k most frequent. Because the root is the smallest, draining the heap yields ascending frequency, so the output array is filled backwards.

Solution

```java
public static int[] topKFrequent(int[] a, int k) {
    Map<Integer,Integer> freq = new HashMap<>();
    for (int x : a) freq.merge(x, 1, Integer::sum);

    PriorityQueue<Map.Entry<Integer,Integer>> heap =
        new PriorityQueue<>(Map.Entry.comparingByValue());   // min-heap
    for (Map.Entry<Integer,Integer> e : freq.entrySet()) {
        heap.offer(e);
        if (heap.size() > k) heap.poll();     // drop the least frequent
    }
    int[] out = new int[heap.size()];
    for (int i = out.length - 1; i >= 0; i--) out[i] = heap.poll().getKey();
    return out;
}
```

Cost. $O(n \log k)$ time, $O(n)$ space for the counts.

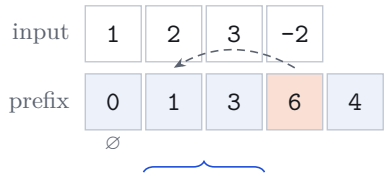
Trap. Filling the output forwards. The heap drains smallest-first, so a forward fill returns the k most frequent in exactly the wrong order.

P21. Subarray sum equals k

Medium

Count the contiguous subarrays summing to exactly k. Values may be negative.

Idea. Prefix sums. A window sums to k whenever an earlier prefix equals the current prefix minus k.



$6 - 1 = 5$: the window summing to 5 is $2 + 3$

How it works. Let P_i be the sum of the first i elements. The sum of the window $(j, i]$ is $P_i - P_j$. Asking for a window summing to k is therefore asking: how many earlier prefixes equal $P_i - k$? Keep a map from prefix value to how many times it has occurred, and the answer accumulates in one pass.

The map is seeded with $0 \mapsto 1$ for the empty prefix, which is what lets a window starting at index 0 be counted.

It is worth knowing why a sliding window fails here. Windows work when growing the window can only increase the sum, so you can shrink from the left when you overshoot. Negative values destroy that monotonicity; prefix sums do not care.

Solution

```
public static int subarraysSummingTo(int[] a, int k) {
    Map<Long,Integer> counts = new HashMap<>();
    counts.put(0L, 1);                      // the empty prefix
    long prefix = 0;
    int total = 0;
    for (int x : a) {
        prefix += x;
        total += counts.getOrDefault(prefix - k, 0);
        counts.merge(prefix, 1, Integer::sum);
    }
    return total;
}
```

Cost. $O(n)$ time, $O(n)$ space.

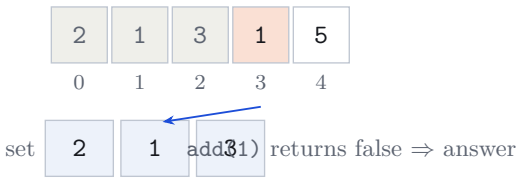
Trap. Reaching for a sliding window because the problem says "contiguous". Check for negative values first — they are the tell.

P22. First duplicate

Easy

Return the first value that appears twice, scanning left to right, or -1.

Idea. A `HashSet`. `add` returns `false` when the element was already there, so the check and the insert are a single call.



How it works. `Set.add` is specified to return whether the set changed. That single boolean replaces a `contains` followed by an `add`, halving the number of hash lookups and removing the chance of writing one without the other.

Be sure which question is being asked. This returns the value whose *second* occurrence comes earliest. "The duplicate whose first occurrence is earliest" is a different answer and needs a second pass.

Solution

```java
public static int firstDuplicate(int[] a) {
    Set<Integer> seen = new HashSet<>();
    for (int x : a) if (!seen.add(x)) return x;
    return -1;
}
```

Cost. $O(n)$ time, $O(n)$ space.

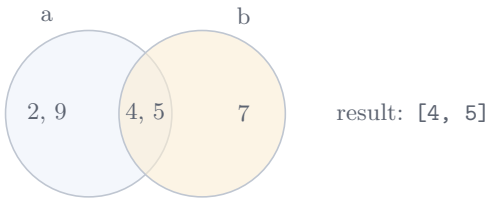
Trap. Misreading which "first" is wanted. Also, -1 as a sentinel is wrong if -1 can itself be a duplicate value — return `Integer` and `null` if the statement allows negatives.

P23. Intersection of two arrays

Easy

Return the values present in both arrays, each once.

Idea. Put one array in a set, filter the other against it, and collect the hits into a second set so duplicates collapse.



How it works. The first set turns "is this value in `a`" from a scan into an $O(1)$ question, which is the whole saving over the nested loop. The second set exists for a different reason: if `b` contains a shared value twice, a plain list would emit it twice.

`LinkedHashSet` is used rather than `HashSet` so the output order is the order of first appearance in `b`. Plain `HashSet` leaves the order unspecified, and graders usually compare arrays exactly.

Solution

```java
public static int[] intersection(int[] a, int[] b) {
    Set<Integer> inA = new HashSet<>();
    for (int x : a) inA.add(x);
    Set<Integer> both = new LinkedHashSet<>();
    for (int x : b) if (inA.contains(x)) both.add(x);
    int[] out = new int[both.size()];
    int i = 0;
    for (int x : both) out[i++] = x;
    return out;
}
```

Cost. $O(n + m)$ time, $O(n)$ space.

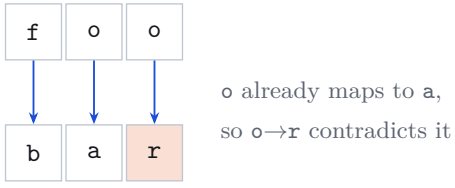
Trap. Collecting into a `List`, which repeats values that recur in `b`.

P24. Isomorphic strings

Medium

Two strings are isomorphic if the characters of one map onto the characters of the other consistently and one-to-one. `"egg"` and `"add"` are; `"foo"` and `"bar"` are not.

Idea. Two maps, one in each direction. One map alone permits two different characters to land on the same target.



How it works. Consistency in one direction says: the same source character always produces the same target. One-to-one adds the reverse: no two source characters share a target. `"ab"` and `"cc"` satisfies the first and fails the second, which is why a single map is not enough.

`putIfAbsent` is doing double duty: it records the mapping if it is new and returns the existing one if it is not, so the check and the write are one call per direction.

Solution

```
public static boolean isIsomorphic(String s, String t) {
    if (s == null || t == null || s.length() != t.length()) return false;
    Map<Character,Character> forward = new HashMap<>();
    Map<Character,Character> backward = new HashMap<>();
    for (int i = 0; i < s.length(); i++) {
        char a = s.charAt(i), b = t.charAt(i);
        Character f = forward.putIfAbsent(a, b);
        Character g = backward.putIfAbsent(b, a);
        if ((f != null && f != b) || (g != null && g != a)) return false;
    }
    return true;
}
```

Cost. $O(n)$ time, $O(1)$ space for a bounded alphabet.

Trap. Comparing the boxed `Character` values with `!=` works here only because the cache covers ASCII. With `Integer` keys above 127 the identical code fails silently — make unboxing a habit.

5. Sorting, intervals and comparators

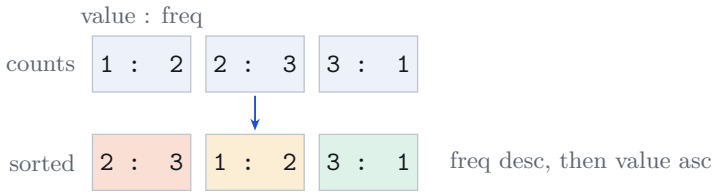
Seven problems. Writing a correct multi-key comparator under time pressure is a skill in itself, and interval sweeps are the most common medium-difficulty question in a screen.

P25. Sort by frequency, then by value

Medium

More frequent values come first; equally frequent values come in ascending numeric order. `[1,1,2,2,2,3]` gives `[2,2,2,1,1,3]`.

Idea. Count first, then sort the *boxed* values with a two-key comparator.



How it works. The comparator reads as two questions asked in order. Are the frequencies different? Then that decides it, descending — note the arguments reversed, `compare(fy, fx)`, which is how you get descending from an ascending comparison. Otherwise fall through to the value, ascending.

The practical obstacle is that `Arrays.sort(int[], Comparator)` does not exist. Primitive arrays cannot take a comparator at all, so you must box into `Integer[]` or sort a `List<Integer>`. Discovering this at minute 50 of a 70-minute test is a bad time to find out.

Solution

```
public static int[] sortByFrequency(int[] a) {
    Map<Integer,Integer> freq = new HashMap<>();
    for (int x : a) freq.merge(x, 1, Integer::sum);

    Integer[] boxed = new Integer[a.length];
    for (int i = 0; i < a.length; i++) boxed[i] = a[i];

    Arrays.sort(boxed, (x, y) -> {
        int fx = freq.get(x), fy = freq.get(y);
        if (fx != fy) return Integer.compare(fy, fx);   // frequency desc
        return Integer.compare(x, y);                   // value asc
    });

    int[] out = new int[a.length];
    for (int i = 0; i < a.length; i++) out[i] = boxed[i];
    return out;
}
```

Cost. $O(n \log n)$ time, $O(n)$ space.

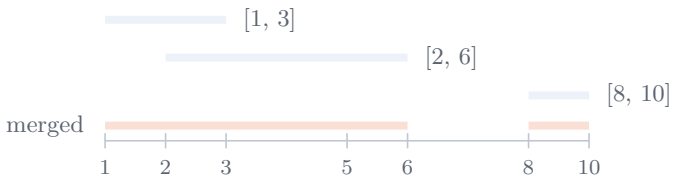
Trap. Writing `fy - fx` instead of `Integer.compare(fy, fx)`. The subtraction overflows for large values and silently inverts the order.

P26. Merge overlapping intervals

Medium

Merge every pair of intervals that overlaps. `[[1,3],[2,6],[8,10]]` gives `[[1,6],[8,10]]`.

Idea. Sort by start. Then one pass: either the next interval starts inside the one you are building, so stretch it, or it does not, so close this one and open a new one.



How it works. Sorting by start is what makes a single pass possible: afterwards, anything that overlaps the interval you are currently building must come next, because it cannot start earlier than something you have already passed.

From there each new interval is one of two cases. If its start is at or before the current end, the two overlap and merge into one whose end is the *later* of the two. If its start is beyond the current end, there is a gap, so the current interval is finished and goes to the output.

Solution

```java
public static int[][] merge(int[][] intervals) {
    if (intervals == null || intervals.length == 0) return new int[0][];
    int[][] a = intervals.clone();
    Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));

    List<int[]> out = new ArrayList<>();
    int[] current = new int[] {a[0][0], a[0][1]};
    for (int i = 1; i < a.length; i++) {
        if (a[i][0] <= current[1]) {
            current[1] = Math.max(current[1], a[i][1]);
        } else {
            out.add(current);
            current = new int[] {a[i][0], a[i][1]};
        }
    }
    out.add(current);
    return out.toArray(new int[0][]);
}
```

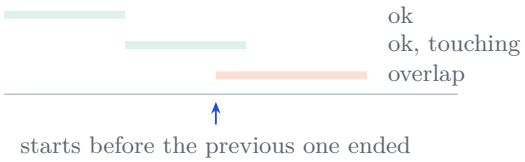
Cost. $O(n \log n)$ time, $O(n)$ space.

Trap. `current[1] = a[i][1]` without `Math.max` shrinks the interval when one is fully contained in another: merging `[1,10]` with `[2,3]` must still end at 10. And the final `out.add(current)` after the loop is easy to forget — without it you lose the last interval every time.

P27. Can one person attend all meetings? Easy

Return `true` if no two meetings overlap.

Idea. Sort by start; any overlap is then between neighbours.



How it works. After sorting, a conflict can only be between consecutive meetings. If meeting i starts before meeting $i-1$ ends, they clash; and if no consecutive pair clashes then no pair does at all, because any later meeting starts no earlier than its predecessor.

Solution

```java
public static boolean canAttendAll(int[][] meetings) {
    if (meetings == null || meetings.length < 2) return true;
    int[][] a = meetings.clone();
    Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
```

```
    for (int i = 1; i < a.length; i++)
        if (a[i][0] < a[i - 1][1]) return false;
    return true;
}
```

Cost. $O(n \log n)$ time.

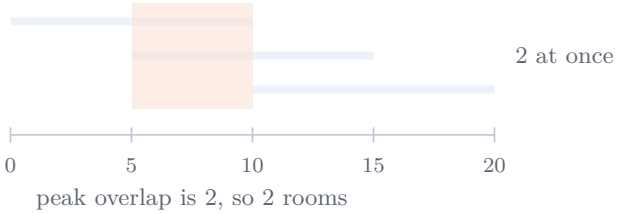
Trap. Whether touching counts as overlapping. `[1,5]` and `[5,9]` do not clash when the end is exclusive — hence `<` and not `<=`. The statement decides; read it twice.

P28. Minimum number of meeting rooms

Medium

Return the largest number of meetings running at the same time.

Idea. A sweep. Sort all the starts and all the ends independently, then walk them: a start raises the count, an end lowers it. Track the peak.



How it works. The pairing between a particular start and its own end is irrelevant to the answer — only the number of events in flight matters. That is why the two arrays are sorted *independently*, which surprises people the first time they see it.

Walk the starts in order. Before counting each one, release every meeting that has already ended by that time. The counter then holds the number of concurrent meetings at that instant, and the running maximum is the number of rooms.

The comparison `ends[e] <= starts[s]` is the back-to-back rule: a room freed at 5 is available to a meeting starting at 5. Using `<` books a needless second room.

Solution

```
public static int minRooms(int[][] meetings) {
    if (meetings == null || meetings.length == 0) return 0;
    int n = meetings.length;
    int[] starts = new int[n], ends = new int[n];
    for (int i = 0; i < n; i++) {
        starts[i] = meetings[i][0];
        ends[i]   = meetings[i][1];
    }
    Arrays.sort(starts);
    Arrays.sort(ends);

    int rooms = 0, peak = 0, e = 0;
    for (int s = 0; s < n; s++) {
        while (e < n && ends[e] <= starts[s]) { rooms--; e++; }
        rooms++;
        peak = Math.max(peak, rooms);
    }
    return peak;
}
```

Cost. $O(n \log n)$ time, $O(n)$ space.

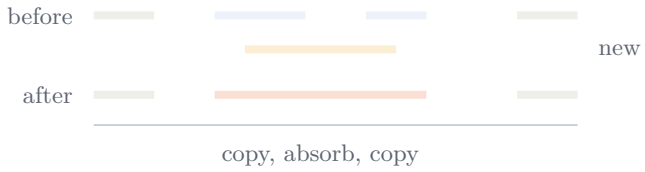
Trap. Trying to keep starts and ends paired. Once you accept that they can be separated, the problem collapses to counting.

P29. Insert an interval

Medium

The intervals are already sorted and non-overlapping. Insert a new one, merging where it overlaps.

Idea. Three phases and no sorting: copy what ends before the new one starts, absorb everything that overlaps into a widened interval, copy the rest.



How it works. Because the input is already sorted, the intervals split into three consecutive blocks: those entirely to the left of the new one, those touching it, and those entirely to the right. Each block is handled by its own loop, and none of them needs to look back.

The middle loop widens `start` and `end` to cover every overlapping interval it consumes. The merged interval is added exactly once, after that loop finishes — adding it inside would emit one copy per overlap.

Solution

```java
public static int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> out = new ArrayList<>();
    int i = 0, n = intervals.length;
    int start = newInterval[0], end = newInterval[1];

    while (i < n && intervals[i][1] < start) out.add(intervals[i++]);

    while (i < n && intervals[i][0] <= end) {
        start = Math.min(start, intervals[i][0]);
        end   = Math.max(end,   intervals[i][1]);
        i++;
    }
    out.add(new int[] {start, end});

    while (i < n) out.add(intervals[i++]);
    return out.toArray(new int[0][]);
}
```

Cost. $O(n)$ time. Re-sorting would waste the guarantee you were given.

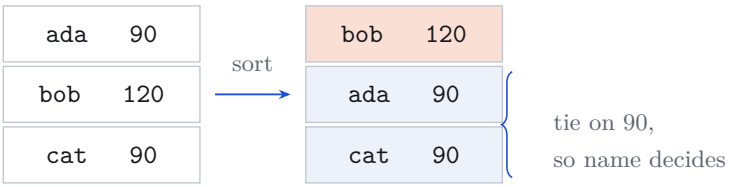
Trap. Adding the merged interval inside the absorb loop.

P30. Sort employees by salary then name

Easy

Salary descending, alphabetical for equal salaries. A pure comparator drill — this exact shape appears in data-processing tasks.

Idea. `Comparator` factory methods compose. Build the ordering declaratively rather than hand-rolling the branches.



How it works. A multi-key ordering is a sequence of questions. Ask the first; if it gives a verdict, stop. Otherwise ask the next. The fluent form expresses exactly that, and the hand-written form below it is the same logic spelled out — worth knowing both, because under pressure the fluent API is easy to half-remember.

Solution

```java
record Employee(String name, int salary) {}

public static void sortEmployees(List<Employee> staff) {
    staff.sort(Comparator.comparingInt(Employee::salary).reversed()
                         .thenComparing(Employee::name));
}

// the hand-written equivalent
public static void sortEmployeesManually(List<Employee> staff) {
    staff.sort((a, b) -> {
        if (a.salary() != b.salary())
            return Integer.compare(b.salary(), a.salary());
        return a.name().compareTo(b.name());
    });
}
```

Cost. $O(n \log n)$. `List.sort` on objects is a stable merge sort, so equal elements keep their input order — which is why the tie-break is only needed when you want a *specific* order, not just a deterministic one.

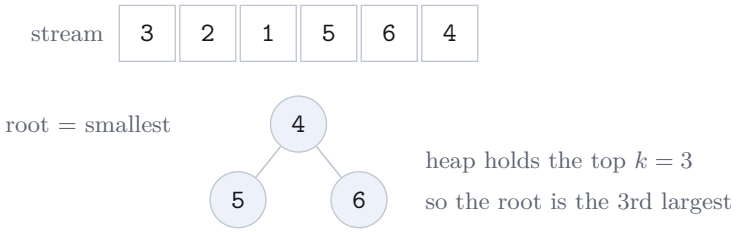
Trap. `.reversed()` reverses everything composed so far. Chaining it after `thenComparing(name)` reverses the name order too. Put it on the salary key alone, as above.

P31. Kth largest element

Medium

Return the kth largest value in an unsorted array, duplicates counted.

Idea. Keep a min-heap of size k. Once everything has been seen, its root is the kth largest overall.



How it works. You never need the whole array ordered — only the boundary between the top k and everything else. A min-heap capped at k maintains exactly that boundary: push each element, and if the heap grows past k, evict its root, which is the smallest thing currently held.

Whatever is left at the end is the k largest values, and the smallest of those — the root — is the kth largest.

Solution

```java
public static int kthLargest(int[] a, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();   // min-heap
    for (int x : a) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

Cost. $O(n \log k)$ time, $O(k)$ space. Sorting is $O(n \log n)$ and perfectly acceptable when k is close to n.

Trap. Reaching for a max-heap. It works, but it holds all n elements, which defeats the point when the input is a stream or n is large.

6. Matrices

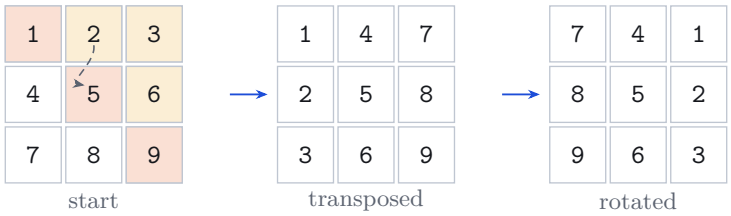
Six problems. Two-dimensional arrays are mostly an index-discipline test: get `rows` and `cols` the right way round and most of the difficulty disappears.

P32. Rotate a square matrix 90 degrees clockwise

Medium

Rotate in place.

Idea. Transpose — reflect across the main diagonal — then reverse each row. Two simple passes beat one confusing index formula.



How it works. The transpose swaps `m[i][j]` with `m[j][i]`, which handles two cells at once. That is why the inner loop starts at `j = i + 1`: it visits only the strict upper triangle, shaded above. Starting at `j = 0` would visit each pair twice and undo every swap, leaving the matrix unchanged — a bug with no exception and no symptom except a wrong answer.

The diagonal is excluded too, correctly: `m[i][i]` swaps with itself.

After the transpose each row holds a column of the original, but in the wrong direction, so reversing every row finishes the rotation.

Solution

```
public static void rotate90(int[][] m) {
    int n = m.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {      // strict upper triangle
            int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
        }
    for (int[] row : m)
        for (int i = 0, j = n - 1; i < j; i++, j--) {
            int t = row[i]; row[i] = row[j]; row[j] = t;
        }
}
```

In the second loop, `row` is a reference to the real row, so writing `row[i]` modifies the matrix. The condition `i < j` rather than `i <= j` skips the pointless self-swap at the centre of an odd-length row.

Cost. $O(n^2)$ time, $O(1)$ space.

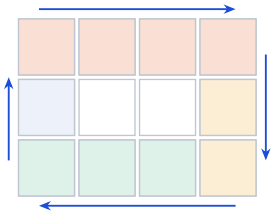
Trap. `j = 0` in the transpose loop. For counter-clockwise, reverse the rows *first* and then transpose.

P33. Spiral traversal

Medium

Return the elements in spiral order.

Idea. Four moving boundaries. Walk an edge, shrink that boundary, and check before the two return edges that the boundaries have not crossed.



1. top row, left to right
2. right column, down
3. bottom row, right to left
4. left column, up

How it works. Keep four indices — `top`, `bottom`, `left`, `right` — naming the rectangle still to be walked. Each edge is traversed and then its boundary moves inwards by one, shrinking the rectangle. When the boundaries cross, everything has been emitted.

The two guards exist because of odd shapes. If the remaining rectangle is a single row, the top-row pass already emitted it; walking the bottom row would emit it again, backwards. `if (top <= bottom)` prevents that, and the symmetric guard handles a single column.

Solution

```java
public static List<Integer> spiral(int[][] m) {
    List<Integer> out = new ArrayList<>();
    if (m == null || m.length == 0) return out;
    int top = 0, bottom = m.length - 1;
    int left = 0, right = m[0].length - 1;

    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; c++) out.add(m[top][c]);
        top++;
        for (int r = top; r <= bottom; r++) out.add(m[r][right]);
        right--;
        if (top <= bottom) {
            for (int c = right; c >= left; c--) out.add(m[bottom][c]);
            bottom--;
        }
        if (left <= right) {
            for (int r = bottom; r >= top; r--) out.add(m[r][left]);
            left++;
        }
    }
    return out;
}
```

Cost. $O(\text{rows} \times \text{cols})$ time.

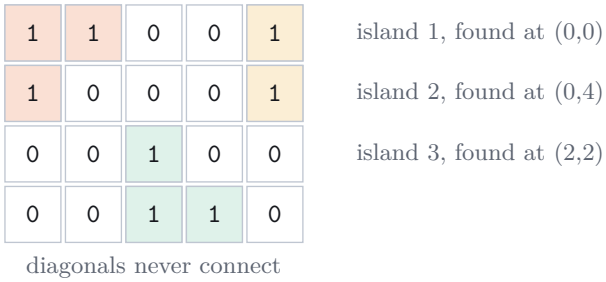
Trap. Omitting the guards. Square matrices hide the bug; a $1 \times n$ or $n \times 1$ input exposes it immediately.

P34. Count islands

Medium

`'1'` is land, `'0'` is water. Count the connected groups of land, four-way adjacency.

Idea. Scan every cell. The first time you touch land, increment the count and flood the whole island so it is never counted again.



How it works. Two loops doing different jobs. The outer double loop is a *scan*: it visits every cell once looking for land nobody has reached. The inner `while` is a *flood*: it erases the entire island the scan just landed on.

The erasing is what makes the counting correct. After an island is flooded, all of its cells are `'0'`, so when the scan later walks over them the `continue` on line 8 skips them. The counter therefore fires once per island, not once per land cell.

Note where the marking happens: a cell is set to `'0'` as it is *pushed*, not when it is popped. That keeps the invariant "a cell is in the queue at most once". Marking on pop instead lets a cell be pushed by each of its neighbours before its first pop — up to four copies each, so four times the memory and work. Bounded, but it is the usual cause of a memory-limit failure on a large grid.

Solution

```
1  public static int countIslands(char[][] grid) {
2      if (grid == null || grid.length == 0) return 0;
3      int rows = grid.length, cols = grid[0].length, islands = 0;
4      int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};
5
6      for (int r = 0; r < rows; r++)
7          for (int c = 0; c < cols; c++) {
8              if (grid[r][c] != '1') continue;
9              islands++;
10             Deque<int[]> queue = new ArrayDeque<>();
11             queue.add(new int[] {r, c});
12             grid[r][c] = '0';                    // mark on push
13             while (!queue.isEmpty()) {
14                 int[] cell = queue.poll();
15                 for (int[] d : dirs) {
16                     int nr = cell[0] + d[0], nc = cell[1] + d[1];
17                     if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
18                     if (grid[nr][nc] != '1') continue;
19                     grid[nr][nc] = '0';
20                     queue.add(new int[] {nr, nc});
21                 }
22             }
23         }
24     return islands;
25 }
```

The bounds check must come before the value check — Java evaluates in order, so reading `grid[nr][nc]` first throws on any edge cell.

Cost. $O(\text{rows} \times \text{cols})$ time and space.

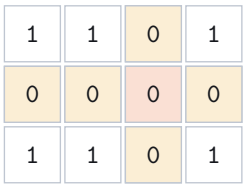
Trap. This function is *destructive*: it overwrites the caller's grid with water. That is a deliberate trick — the input doubles as the visited set — but if the caller needs the grid afterwards, copy it first or use a separate `boolean[][]`.

P35. Set matrix zeroes

Medium

If any cell is zero, set its whole row and column to zero, without a second matrix.

Idea. Two passes. First record *which* rows and columns must be cleared; only then clear them.



one zero clears its row and its column

record first, clear second — or the zeros cascade

How it works. The naive one-pass version fails because the zeros it writes are indistinguishable from the original ones. The pass continues, reads a zero it just wrote, and clears that row too, until the whole matrix is zero.

Separating detection from mutation fixes it. Two boolean arrays remember the rows and columns to clear, and nothing is written until every original zero has been seen.

Solution

```java
public static void setZeroes(int[][] m) {
    if (m == null || m.length == 0) return;
    int rows = m.length, cols = m[0].length;
    boolean[] clearRow = new boolean[rows];
    boolean[] clearCol = new boolean[cols];

    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            if (m[r][c] == 0) { clearRow[r] = true; clearCol[c] = true; }

    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            if (clearRow[r] || clearCol[c]) m[r][c] = 0;
}
```

Cost. $O(\text{rows} \times \text{cols})$ time, $O(\text{rows} + \text{cols})$ space.

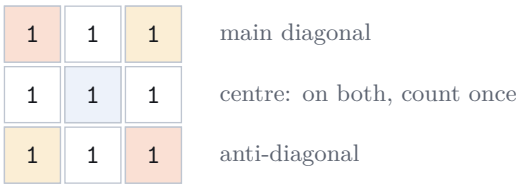
Trap. The $O(1)$-space variant stores the flags in row 0 and column 0 and needs a separate flag for each of those. It is more clever than it is worth under time pressure — write the two arrays.

P36. Diagonal sums

Easy

Sum the main diagonal and the anti-diagonal of a square matrix, counting the centre of an odd-sized matrix once.

Idea. One loop. At step i the two cells are $m[i][i]$ and $m[i][n-1-i]$; they coincide exactly when $i = n-1-i$.



How it works. Both diagonals have exactly n cells and can be indexed by the same loop variable, so no nested loop is needed. The only subtlety is the shared centre cell of an odd matrix, which belongs to both diagonals and must contribute once — the guard `i != n - 1 - i` handles it.

Solution

```java
public static long diagonalSum(int[][] m) {
    int n = m.length;
    long total = 0;
    for (int i = 0; i < n; i++) {
        total += m[i][i];
        if (i != n - 1 - i) total += m[i][n - 1 - i];
    }
    return total;
}
```

Cost. $O(n)$ time, $O(1)$ space — note this is linear, not quadratic: you never touch the other $n^2 - 2n$ cells.

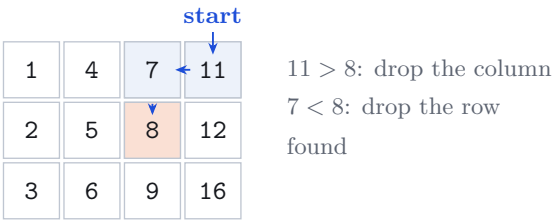
Trap. Double-counting the centre. The total can also exceed `int`.

P37. Search a row-and-column sorted matrix

Medium

Every row is sorted left to right and every column top to bottom. Is the target present?

Idea. Start at the top-right corner. That cell is the largest in its row and the smallest in its column, so one comparison eliminates a whole row or a whole column.



How it works. The top-right corner is the only starting point that gives information. It is the maximum of its row, so if it is larger than the target, the target cannot be anywhere in that column — every cell below is larger still. And it is the minimum of its column, so if it is smaller than the target, nothing in that row can match.

Either way one comparison deletes an entire row or column, so the search finishes in at most rows + cols steps.

Starting at the top-left fails because both moves lead to larger values and nothing can be eliminated. The bottom-left corner works by the same argument as the top-right.

Solution

```
public static boolean contains(int[][] m, int target) {
    if (m == null || m.length == 0) return false;
    int r = 0, c = m[0].length - 1;
    while (r < m.length && c >= 0) {
        if (m[r][c] == target) return true;
        if (m[r][c] > target) c--;      // whole column is too big
        else r++;                       // whole row is too small
    }
    return false;
}
```

Cost. $O(\text{rows} + \text{cols})$ time, $O(1)$ space.

Trap. Binary-searching each row is $O(\text{rows} \log \text{cols})$ and also correct, but it ignores the column ordering. The staircase walk is simpler and faster.

7. Recursion, BFS and DFS

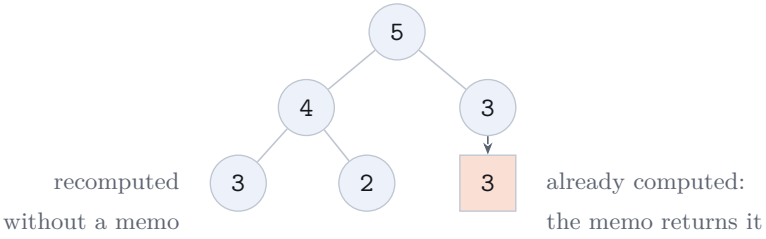
Seven problems. Recursion is fine on trees, where depth is logarithmic. On grids, use an explicit queue — depth there is measured in tens of thousands.

P38. Fibonacci with memoisation

Easy

Return the nth Fibonacci number. The point is the memo, not the formula.

Idea. The naive recursion recomputes the same subproblems exponentially often. Cache each answer, or notice that only two previous values are ever needed and drop the recursion entirely.



How it works. The recursion tree branches twice at every level, so without a cache it does $O(2^n)$ work — and most of it repeats the same handful of subproblems. A memo collapses the tree to a line: each value of n is computed once, then looked up.

The iterative version goes further. Since F_n depends only on the two previous values, there is nothing to store beyond them, and the recursion disappears along with the stack it needed.

Solution

```java
public static long fib(int n) {
    if (n < 2) return n;
    long prev = 0, current = 1;
    for (int i = 2; i <= n; i++) {
        long next = prev + current;
        prev = current;
        current = next;
    }
    return current;
}

// the memoised recursion, when the recurrence is not so easily unrolled
public static long fibMemo(int n, Map<Integer,Long> memo) {
    if (n < 2) return n;
    Long cached = memo.get(n);
    if (cached != null) return cached;
    long value = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    memo.put(n, value);
    return value;
}
```

Cost. Iterative: $O(n)$ time, $O(1)$ space. Memoised: $O(n)$ both, plus the call stack.

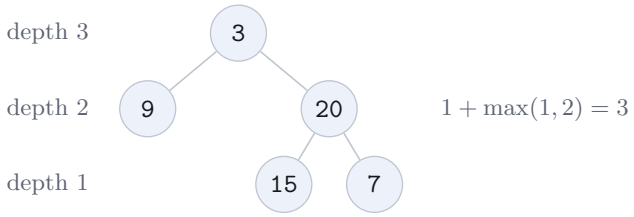
Trap. `int` overflows at `fib(47)`. If the statement does not bound n, use `long` and say so out loud.

P39. Maximum depth of a binary tree

Easy

Return the number of nodes on the longest root-to-leaf path.

Idea. The depth of a node is one more than the deeper of its two subtrees.



How it works. This is the template every tree recursion follows, and it has exactly two parts. The base case answers for the empty tree: `null` has depth 0. The recursive case combines the answers from the children — here by taking the larger and adding one for the current node.

Returning 0 rather than 1 for `null` is what makes a leaf come out at depth 1 rather than 2.

Solution

```java
static class Node {
    int value;
    Node left, right;
}

public static int maxDepth(Node root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Cost. $O(n)$ time, $O(h)$ stack for height h — $O(\log n)$ balanced, $O(n)$ for a degenerate chain.

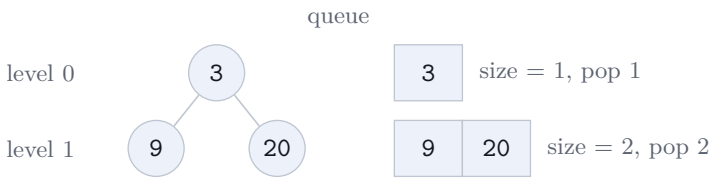
Trap. Returning 1 for `null`, which counts a missing child as a level.

P40. Level-order traversal

Medium

Return the node values grouped by depth, top level first.

Idea. BFS with a queue, taking a whole level at a time: record the queue size *before* the loop, then pop exactly that many nodes.



How it works. A plain BFS visits the nodes in level order but gives you no way to tell where one level ends. The snapshot does: at the top of each iteration the queue contains exactly the current level and nothing else, so its size is that level's width. Popping precisely that many nodes — while pushing their children behind them — consumes one level and leaves the queue holding exactly the next.

Solution

```java
public static List<List<Integer>> levelOrder(Node root) {
    List<List<Integer>> out = new ArrayList<>();
    if (root == null) return out;
    Deque<Node> queue = new ArrayDeque<>();
    queue.add(root);
    while (!queue.isEmpty()) {
        int levelSize = queue.size();            // snapshot
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < levelSize; i++) {
            Node node = queue.poll();
            level.add(node.value);
            if (node.left != null) queue.add(node.left);
            if (node.right != null) queue.add(node.right);
```

```
        }
        out.add(level);
    }
    return out;
}
```

Cost. $O(n)$ time, $O(w)$ space for the widest level w.

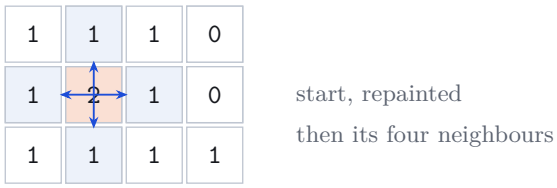
Trap. `for (int i = 0; i < queue.size(); i++)` re-evaluates the size as children are added, so levels bleed into one another. Take the snapshot into a variable.

P41. Flood fill

Easy

Repaint a pixel and every same-coloured pixel connected to it, four-way.

Idea. BFS from the start pixel. Guard against the new colour equalling the old one, or the fill never terminates.



How it works. Identical in shape to counting islands: a queue, four offsets, mark on push. The one addition is the guard at the top.

If the new colour equals the old one, then repainting a cell leaves it still matching the target, so it qualifies to be enqueued again — by every neighbour, forever. The early return costs one comparison and removes an infinite loop.

Solution

```
public static int[][] floodFill(int[][] image, int sr, int sc, int colour) {
    int old = image[sr][sc];
    if (old == colour) return image;              // the essential guard
    int rows = image.length, cols = image[0].length;
    int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};

    Deque<int[]> queue = new ArrayDeque<>();
    queue.add(new int[] {sr, sc});
    image[sr][sc] = colour;
    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        for (int[] d : dirs) {
            int r = cell[0] + d[0], c = cell[1] + d[1];
            if (r < 0 || r >= rows || c < 0 || c >= cols) continue;
            if (image[r][c] != old) continue;
            image[r][c] = colour;
            queue.add(new int[] {r, c});
        }
    }
    return image;
}
```

Cost. $O(\text{rows} \times \text{cols})$ time and space.

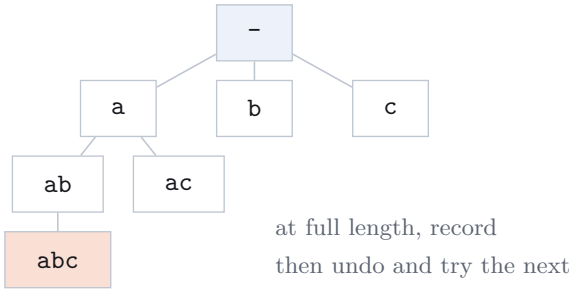
Trap. Omitting the `old == colour` guard. It only hangs when the caller repaints with the colour already there, which is exactly the test case a grader includes.

Also appears as: The engine behind Task 3 of the assessment.

P42. All permutations of a string

Medium

Return every ordering of the characters. There are $n!$, so this is only ever asked for small inputs.
Idea. Backtracking. Choose an unused character, recurse, then put it back.



How it works. The search walks down the tree making a choice at each level, and when the current string reaches full length it records a permutation. The two `undo` lines at the bottom of the loop are what let it walk back *up* and explore a different branch.

Both the `StringBuilder` and the `used` array are shared across the entire search — there is only one of each. That is why every change must be reversed on the way out: whatever state a branch leaves behind, the next branch inherits.

Solution

```
public static List<String> permutations(String s) {
    List<String> out = new ArrayList<>();
    if (s == null || s.isEmpty()) return out;
    build(s.toCharArray(), new boolean[s.length()],
          new StringBuilder(), out);
    return out;
}

private static void build(char[] chars, boolean[] used,
                          StringBuilder current, List<String> out) {
    if (current.length() == chars.length) {
        out.add(current.toString());
        return;
    }
    for (int i = 0; i < chars.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        current.append(chars[i]);
        build(chars, used, current, out);
        current.deleteCharAt(current.length() - 1);   // undo
        used[i] = false;                              // undo
    }
}
```

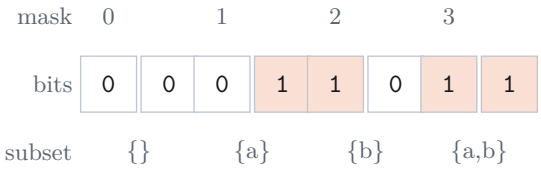
Cost. $O(n \cdot n!)$ time.

Trap. Forgetting either undo. You get a fraction of the permutations, or garbage, and the output is long enough that the bug is hard to spot by eye.

P43. All subsets

Medium

Return every subset of a set of distinct numbers — 2^n of them.
Idea. Each element is either in or out, so the subsets correspond exactly to the binary numbers from 0 to $2^n - 1$. No recursion needed.



How it works. Counting from 0 to $2^n - 1$ enumerates every possible combination of in/out decisions exactly once, with no bookkeeping. Bit i of the counter says whether element i is included, and `(mask & (1 << i)) != 0` tests it.

This is shorter and faster than the recursive version, and there is no undo step to forget.

Solution

```java
public static List<List<Integer>> subsets(int[] a) {
    List<List<Integer>> out = new ArrayList<>();
    int total = 1 << a.length;                    // 2^n
    for (int mask = 0; mask < total; mask++) {
        List<Integer> subset = new ArrayList<>();
        for (int i = 0; i < a.length; i++)
            if ((mask & (1 << i)) != 0) subset.add(a[i]);
        out.add(subset);
    }
    return out;
}
```

Cost. $O(n \cdot 2^n)$ time.

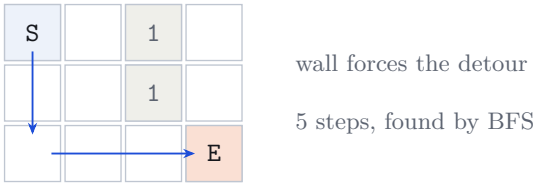
Trap. `1 << a.length` overflows once the array reaches 31 elements. Academic here — 2^{31} subsets would not fit in memory — but the shift is what breaks first.

P44. Shortest path in a grid with obstacles

Medium

Cells are open (`0`) or blocked (`1`). Fewest steps from the top-left to the bottom-right, four-way, or -1.

Idea. BFS, not DFS. On an unweighted graph BFS reaches every cell by a shortest path, so the first arrival at the target is the answer.



How it works. BFS expands in rings: everything one step away, then everything two steps away, and so on. So the first time the target is dequeued, no shorter route to it exists — that is a property of the traversal order, not something the code has to check.

Counting the steps uses the same level-snapshot trick as the tree traversal: each pass of the outer loop consumes one ring and increments the distance.

DFS would also find *a* path, but not the shortest, and people reach for it because recursion is quicker to type. If the question says shortest and the edges are unweighted, it is BFS.

Solution

```java
public static int shortestPath(int[][] grid) {
    if (grid == null || grid.length == 0) return -1;
    int rows = grid.length, cols = grid[0].length;
    if (grid[0][0] == 1 || grid[rows - 1][cols - 1] == 1) return -1;
    int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};

    boolean[][] seen = new boolean[rows][cols];
    Deque<int[]> queue = new ArrayDeque<>();
    queue.add(new int[] {0, 0});
    seen[0][0] = true;
    int steps = 1;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
```

```
        for (int i = 0; i < levelSize; i++) {
            int[] cell = queue.poll();
            if (cell[0] == rows - 1 && cell[1] == cols - 1) return steps;
            for (int[] d : dirs) {
                int r = cell[0] + d[0], c = cell[1] + d[1];
                if (r < 0 || r >= rows || c < 0 || c >= cols) continue;
                if (seen[r][c] || grid[r][c] == 1) continue;
                seen[r][c] = true;
                queue.add(new int[] {r, c});
            }
        }
        steps++;
    }
    return -1;
}
```

Cost. $O(\text{rows} \times \text{cols})$ time and space.

Trap. Marking `seen` on pop rather than on push, which lets the same cell be queued by several neighbours. Same rule as every other grid search in this book.

8. Stacks and queues

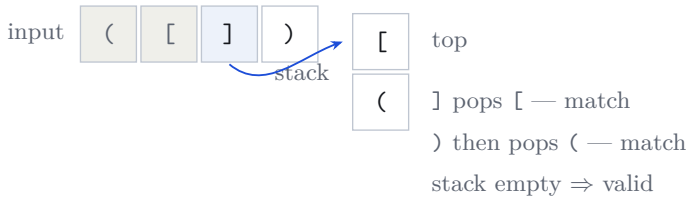
Five problems. A stack is the right answer whenever the most recent unresolved thing is the one you need next.

P45. Valid parentheses

Easy

Given a string of `()`, `[]` and `{}`, decide whether every bracket is closed by the matching kind, in the right order.

Idea. Push openers. On a closer, the top of the stack must be its partner. At the end the stack must be empty.



How it works. Nesting is last-in, first-out by definition: the bracket you must close next is always the most recent one you opened. That is precisely what a stack models, which is why no other data structure is needed.

Two endings are easy to forget. A closer arriving when the stack is empty means there was never a matching opener, so `isEmpty()` must be checked before `pop()` or it throws. And a non-empty stack at the end means openers were never closed, which is also invalid — hence the return value is `stack.isEmpty()`, not `true`.

Solution

```
public static boolean isValid(String s) {
    if (s == null) return true;
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(' : case '[' : case '{' : stack.push(c); break;
            case ')' : if (stack.isEmpty() || stack.pop() != '(') return false; break;
            case ']' : if (stack.isEmpty() || stack.pop() != '[') return false; break;
            case '}' : if (stack.isEmpty() || stack.pop() != '{') return false; break;
            default  : break;    // ignore anything else
        }
    }
    return stack.isEmpty();
}
```

Short-circuit evaluation does the work: `stack.isEmpty() || stack.pop() != '('` never calls `pop` on an empty stack.

Cost. $O(n)$ time, $O(n)$ space in the worst case of all openers.

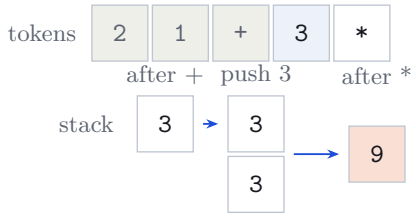
Trap. Counting brackets instead of stacking them. `"([)]"` has equal counts of each kind and is invalid; only a stack catches the ordering.

P46. Evaluate reverse Polish notation

Medium

Evaluate tokens like `["2","1","+","3","*"]`, meaning $(2 + 1) \times 3 = 9$.

Idea. Push numbers. On an operator, pop two, apply, push the result.



How it works. Postfix notation needs no precedence rules and no parentheses, because the order of the tokens already encodes the order of evaluation. Every operator applies to the two most recently produced values, which is exactly what sits on top of the stack.

The order of the two pops matters. The first value popped is the *right-hand* operand, because it was pushed last. For `+` and `*` the mistake is invisible; for `-` and `/` it gives the wrong sign or the reciprocal, so the bug survives your first two test cases and fails the third.

Solution

```java
public static int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String t : tokens) {
        switch (t) {
            case "+": case "-": case "*": case "/": {
                int right = stack.pop();           // order matters
                int left  = stack.pop();
                stack.push(apply(left, right, t));
                break;
            }
            default:
                stack.push(Integer.parseInt(t));
        }
    }
    return stack.pop();
}

private static int apply(int a, int b, String op) {
    switch (op) {
        case "+": return a + b;
        case "-": return a - b;
        case "*": return a * b;
        default : return a / b;
    }
}
```

Cost. $O(n)$ time, $O(n)$ space.

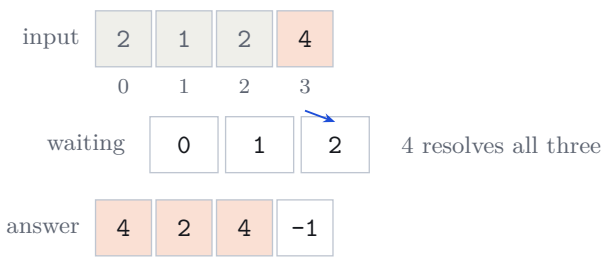
Trap. Popping the operands in the wrong order.

P47. Next greater element

Medium

For every element, find the first element to its right that is larger, or -1. `[2,1,2,4]` gives `[4,2,4,-1]`.

Idea. A monotonic stack of *indices* still waiting for an answer. A new value resolves every waiting index smaller than it.



How it works. The stack holds indices whose answer is not yet known, and it is always in decreasing order of value — that is the invariant that makes it work. When a new value arrives, every index on the stack with a smaller value has just found its next greater element, so each one is popped and written.

The cost argument is the interesting part. The inner `while` looks like it could make this quadratic, but each index is pushed exactly once and popped at most once over the entire run. The total number of inner iterations is therefore bounded by n, so the whole thing is linear — an amortised argument, not a per-step one.

Store indices rather than values, or you cannot write the answer back into the right slot.

Solution

```java
public static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] out = new int[n];
    Arrays.fill(out, -1);
    Deque<Integer> waiting = new ArrayDeque<>();   // indices

    for (int i = 0; i < n; i++) {
        while (!waiting.isEmpty() && a[waiting.peek()] < a[i])
            out[waiting.pop()] = a[i];
        waiting.push(i);
    }
    return out;
}
```

Whatever is left on the stack at the end has no greater element to its right, and those slots keep the -1 from `Arrays.fill`.

Cost. $O(n)$ time amortised, $O(n)$ space.

Trap. The nested-loop version is $O(n^2)$ and reads so naturally that people write it without noticing.

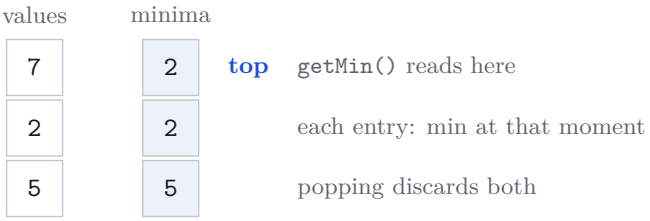
Also appears as: "Daily temperatures", where the answer is the gap `i - j` rather than the value.

P48. Min stack

Medium

A stack that also reports its minimum in $O(1)$.

Idea. A second stack holding the minimum *as of* each push. The two move in lockstep.



How it works. The trick is to store, alongside each value, what the minimum was when that value was pushed. Then popping a value automatically restores the previous minimum — it is sitting right underneath on the second stack.

A single `int min` field cannot do this. It is correct until you pop the minimum itself, at which point the previous minimum is unrecoverable without scanning the whole stack.

Solution

```java
class MinStack {
    private final Deque<Integer> values = new ArrayDeque<>();
    private final Deque<Integer> minima = new ArrayDeque<>();

    void push(int x) {
        values.push(x);
        minima.push(minima.isEmpty() ? x : Math.min(x, minima.peek()));
```

```
    }

    int pop() {
        minima.pop();
        return values.pop();
    }

    int top()     { return values.peek(); }
    int getMin()  { return minima.peek(); }
}
```

Cost. $O(1)$ for every operation, $O(n)$ space — you trade memory for the constant-time minimum.

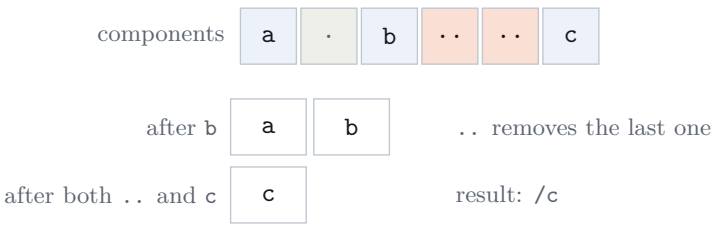
Trap. Pushing onto `minima` only when the new value is a new minimum. That saves space but breaks the lockstep, so `pop` then has to decide whether to pop the second stack, and getting that condition wrong is the usual bug.

P49. Simplify a Unix path

Medium

Turn `"/a/./b/../../c/"` into `"/c"`. Handle `.`, `..` and repeated slashes.

Idea. Split on slashes and push each real component. `..` pops; `.` and empty components are skipped.



How it works. Splitting on `"/"` produces empty strings for leading, trailing and doubled slashes, so those three cases collapse into one test. `.` means "stay here" and is simply skipped. `..` means "go up", which is a pop.

The deque is used as a list here, not a stack: components are appended at the end and removed from the end, but the final join reads them front to back.

`pollLast` rather than `removeLast` is deliberate. At the root there is nothing to go up to, and `..` must do nothing — `poll` returns `null` on an empty deque where `remove` throws.

Solution

```
public static String simplify(String path) {
    Deque<String> parts = new ArrayDeque<>();     // used as a list
    for (String part : path.split("/")) {
        if (part.isEmpty() || part.equals(".")) continue;
        if (part.equals("..")) parts.pollLast();  // no-op when empty
        else parts.addLast(part);
    }
    if (parts.isEmpty()) return "/";
    StringBuilder out = new StringBuilder();
    for (String part : parts) out.append('/').append(part);
    return out.toString();
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. Returning `""` for a path that resolves to the root. The answer is `"/"`.

9. Data processing

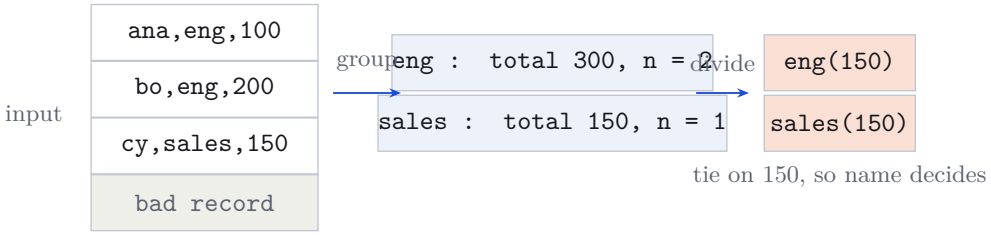
Six problems in the shape candidates actually report from consulting-tech screens: parse records, validate them, group them, aggregate them, report the top few. Less algorithmic than the rest of the book, and more likely to be what you face.

P50. Average salary by department

Medium

Records are `"name,department,salary"`. Return the top n departments by average salary as `"department(average)"`, average descending then name ascending. The average is integer division. Ignore any record without exactly three fields, with an empty department, or whose salary is not a whole number at least zero.

Idea. Accumulate a *total and a count* per department, never a running average. Compute each average once, at the end, from exact numbers.



How it works. Keeping a total and a count rather than a running average is the one design decision that matters. Averaging averages is wrong whenever departments differ in size, and it compounds a rounding error at every step. With a total and a count the division happens exactly once, on exact inputs.

The validation is a ladder of cheap tests applied in order, each one a `continue`. Note that `parseLong` succeeding is not the same as the salary being valid — `-5` parses perfectly well and is still rejected by the statement.

The comparator asks two questions in order: average descending first, then name ascending as the tie-break.

Solution

```
public static String[] topDepartments(String[] records, int n) {
    if (records == null || n <= 0) return new String[0];
    Map<String,long[]> tally = new HashMap<>();      // [total, count]

    for (String record : records) {
        if (record == null) continue;
        String[] parts = record.split(",", -1);      // keep empty fields
        if (parts.length != 3) continue;
        String dept = parts[1].trim();
        if (dept.isEmpty()) continue;
        long salary;
        try { salary = Long.parseLong(parts[2].trim()); }
        catch (NumberFormatException e) { continue; }
        if (salary < 0) continue;

        long[] slot = tally.computeIfAbsent(dept, k -> new long[2]);
        slot[0] += salary;
        slot[1]++;
    }
    if (tally.isEmpty()) return new String[0];

    List<String> names = new ArrayList<>(tally.keySet());
    names.sort((a, b) -> {
```

```
        long avgA = tally.get(a)[0] / tally.get(a)[1];
        long avgB = tally.get(b)[0] / tally.get(b)[1];
        if (avgA != avgB) return Long.compare(avgB, avgA);
        return a.compareTo(b);
    });

    int size = Math.min(n, names.size());
    String[] out = new String[size];
    for (int i = 0; i < size; i++) {
        long[] slot = tally.get(names.get(i));
        out[i] = names.get(i) + "(" + (slot[0] / slot[1]) + ")";
    }
    return out;
}
```

Cost. $O(R + D \log D)$ for R records and D departments.

Trap. Four, and each one is a real submission killer. `split(",")` without `-1` drops trailing empty fields, so `"a,x,"` looks like a two-field record instead of an invalid three-field one. An `int` total overflows. Averaging averages is wrong. And a negative salary parses fine.

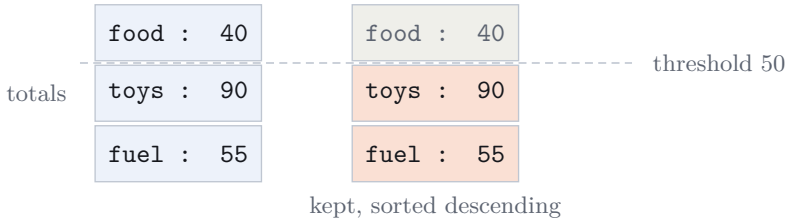
Also appears as: Task 2 of the assessment.

P51. Filter and total transactions

Medium

Records are `"id,category,amount"`. Return the total per category, but only for categories whose total reaches a threshold, sorted by total descending.

Idea. Group fully, *then* filter, then sort. A category only crosses the threshold once all its rows are in.



How it works. The sequence is group, filter, sort — in that order, and the order is the lesson. Filtering inside the parsing loop tests a partial total, so a category that would have qualified on its full total gets dropped the first time it is seen.

`merge(key, value, Long::sum)` is the compact form of "add to the running total, starting from this value if the key is new".

Solution

```
public static List<String> categoriesOver(String[] rows, long threshold) {
    Map<String,Long> totals = new HashMap<>();
    for (String row : rows) {
        if (row == null) continue;
        String[] parts = row.split(",", -1);
        if (parts.length != 3) continue;
        try {
            totals.merge(parts[1].trim(),
                         Long.parseLong(parts[2].trim()), Long::sum);
        } catch (NumberFormatException ignored) { }
    }

    List<Map.Entry<String,Long>> kept = new ArrayList<>();
    for (Map.Entry<String,Long> e : totals.entrySet())
        if (e.getValue() >= threshold) kept.add(e);

```

```
    kept.sort((a, b) -> b.getValue().compareTo(a.getValue()));

    List<String> out = new ArrayList<>();
    for (Map.Entry<String,Long> e : kept)
        out.add(e.getKey() + "=" + e.getValue());
    return out;
}
```

Cost. $O(R + C \log C)$.

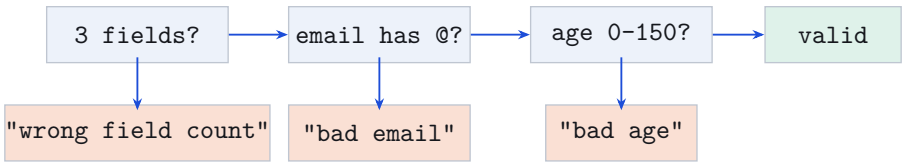
Trap. `(int)(b.getValue() - a.getValue())` as the comparator. The subtraction overflows on large totals and the cast then reverses the order. Use `compareTo` or `Long.compare`.

P52. Validate records and report the errors

Medium

Given `"id,email,age"`, return one message per invalid record in input order: `"line 3: bad age"`. Report only the first problem in each record.

Idea. A validation ladder: check in a fixed order and return the first failure.



How it works. Each check is a gate: pass and fall through to the next, fail and return immediately with that message. Returning on the first failure is what gives "only the first problem per record", and putting the field-count check first is what stops the later checks from indexing past the end of the array.

Splitting the per-record logic into its own method keeps the loop trivial and the line numbering in one place.

Solution

```
public static List<String> validate(String[] rows) {
    List<String> errors = new ArrayList<>();
    for (int i = 0; i < rows.length; i++) {
        String problem = firstProblem(rows[i]);
        if (problem != null) errors.add("line " + (i + 1) + ": " + problem);
    }
    return errors;
}

private static String firstProblem(String row) {
    if (row == null) return "missing record";
    String[] parts = row.split(",", -1);
    if (parts.length != 3) return "wrong field count";
    if (!parts[1].contains("@")) return "bad email";
    int age;
    try { age = Integer.parseInt(parts[2].trim()); }
    catch (NumberFormatException e) { return "bad age"; }
    if (age < 0 || age > 150) return "bad age";
    return null;
}
```

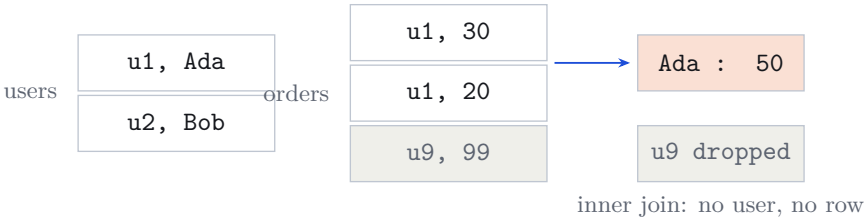
Cost. $O(R)$.

Trap. Line numbers are one-based in every message format anyone asks for, and the loop index is zero-based. At least this one fails every test at once, which makes it easy to spot.

P53. Join two datasets on a key

Medium

One array holds `"userId,name"`, another `"userId,orderTotal"`. Return `"name:total"` for every user present in both, summing each user's orders, sorted by total descending.
Idea. Index the smaller side into a map, then stream the larger side against it. A hash join: $O(n+m)$ where the nested loop is $O(nm)$.



How it works. The map turns "does this user exist" from a scan of the users array into one constant-time lookup, which is the entire saving. The second map accumulates each user's order total as the rows stream past.

The part worth pausing on is which join you were asked for. This one drops orders with no matching user *and* users with no orders. A left join would keep every user, with a total of zero. They give different answers, and the statement will tell you which — read it before you write.

Solution

```java
public static List<String> join(String[] users, String[] orders) {
    Map<String,String> nameById = new HashMap<>();
    for (String u : users) {
        String[] p = u.split(",", -1);
        if (p.length == 2) nameById.put(p[0].trim(), p[1].trim());
    }

    Map<String,Long> totalById = new HashMap<>();
    for (String o : orders) {
        String[] p = o.split(",", -1);
        if (p.length != 2) continue;
        String id = p[0].trim();
        if (!nameById.containsKey(id)) continue;      // inner join
        try { totalById.merge(id, Long.parseLong(p[1].trim()), Long::sum); }
        catch (NumberFormatException ignored) { }
    }

    List<Map.Entry<String,Long>> rows = new ArrayList<>(totalById.entrySet());
    rows.sort((a, b) -> b.getValue().compareTo(a.getValue()));

    List<String> out = new ArrayList<>();
    for (Map.Entry<String,Long> e : rows)
        out.add(nameById.get(e.getKey()) + ":" + e.getValue());
    return out;
}
```

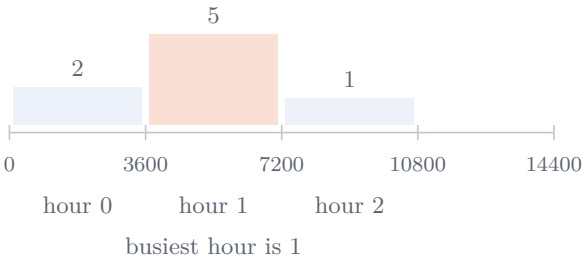
Cost. $O(n+m)$ plus the sort.

Trap. Writing the nested loop. It is correct and it is quadratic; on 10^5 rows each side it will not finish.

P54. Bucket events by hour and find the peak

Medium

Records are `"eventId,timestampSeconds"`. Count the events in each hour and return the busiest hour index, breaking ties by the earlier hour.
Idea. The bucket key is `timestamp / 3600` — integer division *is* the bucketing. Then one scan for the maximum.



How it works. Bucketing by hour is just integer division: every timestamp in $[3600h, 3600(h+1))$ maps to h, and the truncation does it for free.

The scan for the maximum needs its tie-break written out explicitly. The second clause, `e.getValue() == bestCount && e.getKey() < bestHour`, is what makes the earlier hour win. Relying on iteration order instead would be a bug: `HashMap` order is unspecified and changes with the keys, so the answer would vary with the input.

Solution

```
public static long busiestHour(String[] events) {
    Map<Long,Integer> perHour = new HashMap<>();
    for (String e : events) {
        String[] p = e.split(",", -1);
        if (p.length != 2) continue;
        try {
            long hour = Long.parseLong(p[1].trim()) / 3600;
            perHour.merge(hour, 1, Integer::sum);
        } catch (NumberFormatException ignored) { }
    }
    if (perHour.isEmpty()) return -1;

    long bestHour = Long.MAX_VALUE;
    int bestCount = -1;
    for (Map.Entry<Long,Integer> e : perHour.entrySet()) {
        if (e.getValue() > bestCount
                || (e.getValue() == bestCount && e.getKey() < bestHour)) {
            bestCount = e.getValue();
            bestHour = e.getKey();
        }
    }
    return bestHour;
}
```

Cost. $O(R)$.

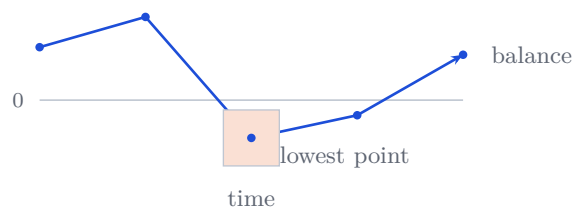
Trap. Trusting `HashMap` iteration order for the tie-break. The tests that catch this are the ones with two equally busy hours.

P55. Running balance per account

Medium

Transactions arrive as `"account,amount"` in chronological order. Return, for each account, the lowest balance it ever reached.

Idea. One pass, two maps. Update the running balance, *then* compare it to the stored minimum.



How it works. The order of the two updates is the whole problem. Apply the transaction to the running balance first, then offer that new balance to the minimum. Comparing before applying would miss a dip caused by the transaction you are currently processing.

`merge(account, now, Math::min)` does the right thing on the first transaction too: with no prior value it simply stores `now`. That is why the minimum is not seeded at zero — an account that only ever receives money has its first deposit as its lowest balance, not 0.

Solution

```
public static Map<String,Long> lowestBalances(String[] transactions) {
    Map<String,Long> balance = new HashMap<>();
    Map<String,Long> lowest  = new HashMap<>();

    for (String t : transactions) {
        String[] p = t.split(",", -1);
        if (p.length != 2) continue;
        String account = p[0].trim();
        long amount;
        try { amount = Long.parseLong(p[1].trim()); }
        catch (NumberFormatException e) { continue; }

        long now = balance.getOrDefault(account, 0L) + amount;
        balance.put(account, now);
        lowest.merge(account, now, Math::min);
    }
    return lowest;
}
```

Cost. $O(R)$ time, $O(A)$ space for A accounts.

Trap. Seeding the minimum at 0 for every account, which silently caps every answer at zero or below.

10. Stretch

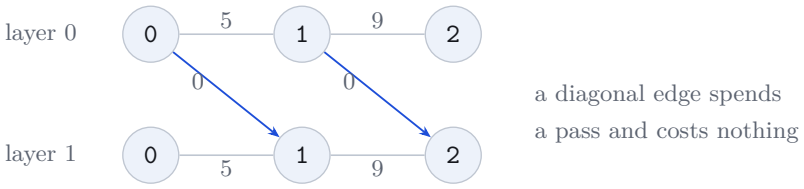
Five problems above the bar. Two of them were cut from the assessment for exactly that reason. Work them when the first eight chapters are comfortable — the point is that a hard problem on the day should feel familiar rather than alarming.

P56. Minimum travel time with k free roads

Hard

Cities $0 \dots n-1$ are joined by bidirectional roads $\{u, v, t\}$. You hold k passes; using one on a road makes it cost 0. Return the least time from city 0 to city $n-1$, or -1.

Idea. The state you search is not a city but a *pair*: the city, and how many passes you have spent. Build $k+1$ copies of the graph and run one Dijkstra over the lot.



How it works. With k passes, knowing only "where am I" is not enough to decide what to do next — you also need "how many passes do I have left". So the state becomes a pair, and the graph becomes $k+1$ stacked copies of itself. Horizontal edges cost their time and stay in the layer; diagonal edges cost nothing and drop one layer down. Dijkstra on this layered graph needs no modification at all.

The first time the queue pops a state at city $n-1$, in any layer, that cost is the answer: Dijkstra pops in non-decreasing cost order, and spending fewer passes is never cheaper, so there is nothing better still to come.

Solution

```
public static int minTravelTime(int n, int[][] roads, int k) {
    if (n == 1) return 0;
    int layers = (int) Math.max(0L, Math.min((long) k, (long) n - 1));

    List<int[]>[] adj = new List[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] r : roads) {
        adj[r[0]].add(new int[] {r[1], r[2]});
        if (r[0] != r[1]) adj[r[1]].add(new int[] {r[0], r[2]});
    }

    long[][] dist = new long[n][layers + 1];
    for (long[] row : dist) Arrays.fill(row, Long.MAX_VALUE);
    dist[0][0] = 0;

    // {cost, city, passesUsed}
    PriorityQueue<long[]> pq =
        new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
    pq.add(new long[] {0, 0, 0});

    while (!pq.isEmpty()) {
        long[] top = pq.poll();
        long cost = top[0];
        int u = (int) top[1], used = (int) top[2];
        if (cost > dist[u][used]) continue;          // stale entry
        if (u == n - 1) return (int) cost;

        for (int[] edge : adj[u]) {
```

```
            int v = edge[0];
            long paid = cost + edge[1];
            if (paid < dist[v][used]) {
                dist[v][used] = paid;
                pq.add(new long[] {paid, v, used});
            }
            if (used < layers && cost < dist[v][used + 1]) {
                dist[v][used + 1] = cost;
                pq.add(new long[] {cost, v, used + 1});
            }
        }
    }
    return -1;
}
```

Cost. $O(E \cdot K \log(V \cdot K))$ with $K = \min(k, n-1) + 1$.

Trap. Three. k can be 10^9, so cap the layers at $n-1$ — a simple path uses no more roads than that, and without the cap you try to allocate a billion rows. Distances overflow `int`. And the tempting greedy — find the shortest path, then zero its k heaviest edges — is simply wrong: with passes in hand the best route is often a different route entirely.

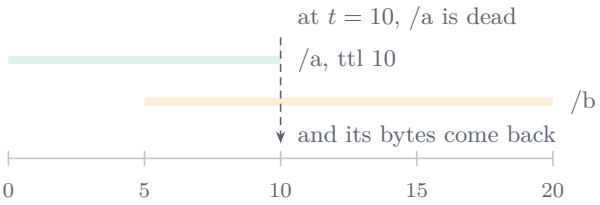
Also appears as: "Cheapest flights with k stops" and "minimum cost path with k refuels" are the same layered-state idea.

P57. Expiring files

Hard

Extend the Part B `CloudStorage` so files can be given a lifetime. `addFileAt(timestamp, user, name, size, ttl)` creates a file alive over $[timestamp, timestamp + ttl)$; the timestamped queries see only files alive at their timestamp; an expired file frees its owner's capacity.

Idea. Make expiry *lazy*. Each timestamped call first deletes everything already dead, through the ordinary delete path, then delegates to the untimed method that already exists.



How it works. The level is designed to attack the incremental `used` counter that made the earlier levels cheap: bytes now free themselves with no operation to hook.

There are two ways out. Recompute live usage on every query — $O(F)$ per call, and it leaves "is this name taken" ambiguous. Or purge lazily, which keeps exactly one notion of "the files that exist" and reuses all four levels below unchanged. Lazy purging wins, and it is amortised $O(1)$ per file because each file is purged once.

Store an *absolute* expiry, not the TTL. A relative lifetime forces you to remember a creation time as well, and makes backup and restore ambiguous about whether a lifetime restarts.

Solution

```
// FileEntry gains: final long expiresAt;   (Long.MAX_VALUE = never)

public Integer addFileAt(int timestamp, String user,
                         String name, int size, int ttl) {
    if (ttl <= 0 || timestamp < 0) return null;
    purge(timestamp);
    // long, because timestamp + ttl overflows int near Integer.MAX_VALUE
    return addEntry(user, name, size, (long) timestamp + (long) ttl);
}
```

```

public Integer getFileSizeAt(int timestamp, String name) {
    purge(timestamp);
    return getFileSize(name);
}

/** Drops every file dead at 'now', crediting each owner's capacity. */
private void purge(long now) {
    List<String> expired = null;
    for (Map.Entry<String,FileEntry> e : files.entrySet())
        if (e.getValue().expiresAt <= now) {
            if (expired == null) expired = new ArrayList<>();
            expired.add(e.getKey());
        }
    if (expired == null) return;
    for (String name : expired) removeEntry(name);   // credits the owner
}
```

Cost. $O(F)$ for a purge that finds work, amortised $O(1)$ per file overall.

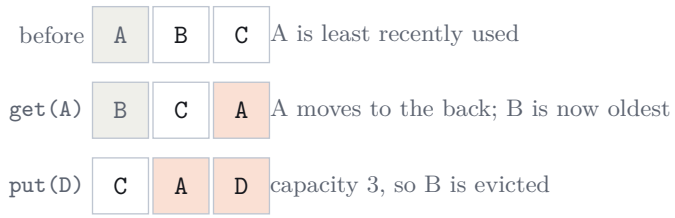
Trap. Deleting from `files` while iterating it throws `ConcurrentModificationException` — collect the dead keys first. And the interval is half-open: a file created at 0 with `ttl` 10 is dead *at* 10, not after it.

P58. LRU cache

Hard

A fixed-capacity cache with $O(1)$ `get` and `put`. When full, the least recently used entry is evicted. A `get` counts as a use.

Idea. `LinkedHashMap` in access-order mode, with `removeEldestEntry` overridden.



How it works. The ordering is the whole data structure. Keep entries in a line from least to most recently used; every access moves an entry to the back; eviction always takes from the front.

`LinkedHashMap` already maintains such a line, and its third constructor argument switches it from insertion order to *access* order, which is what makes a `get` count as a use. `removeEldestEntry` is called after each insertion and evicts the front when it returns true.

Know the manual version too — a `HashMap` from key to node, plus a doubly linked list for the ordering — because interviewers sometimes ban the shortcut. Both are $O(1)$; the library version is twelve lines.

Solution

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    LruCache(int capacity) {
        // initialCapacity, loadFactor, accessOrder=true
        super(16, 0.75f, true);
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
```

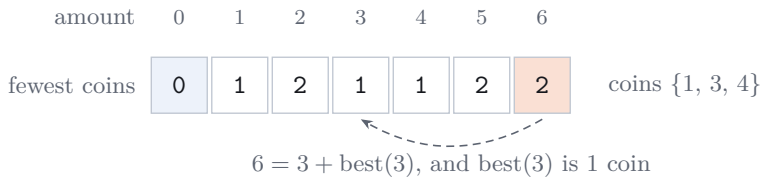
```
    }
}
```

Cost. $O(1)$ expected for both operations.

Trap. The third constructor argument. Left at its default of `false` you get insertion order, which evicts the oldest *inserted* entry — an FIFO cache, not an LRU one. The bug passes every test that never re-reads a key.

P59. Coin change — Hard

Fewest coins summing to a target, or -1. Each denomination may be used any number of times.
Idea. Bottom-up dynamic programming over the amount. The best way to make a is one coin on top of the best way to make $a - \mathit{coin}$.



How it works. Fill the table upwards so every subproblem is already solved when you need it. For amount a, try each coin that fits: using it leaves $a - \mathit{coin}$, whose answer is already in the table, so the candidate is that answer plus one. Take the smallest candidate.

The guard `best[a - coin] == Integer.MAX_VALUE` matters. That sentinel means "unreachable"; adding one to it overflows to a large negative number, which would then look like an excellent answer and poison every later entry.

Solution

```
public static int coinChange(int[] coins, int amount) {
    if (amount < 0) return -1;
    int[] best = new int[amount + 1];
    Arrays.fill(best, Integer.MAX_VALUE);
    best[0] = 0;                                  // zero coins make zero

    for (int a = 1; a <= amount; a++)
        for (int coin : coins) {
            if (coin <= 0 || coin > a) continue;
            if (best[a - coin] == Integer.MAX_VALUE) continue;
            best[a] = Math.min(best[a], best[a - coin] + 1);
        }

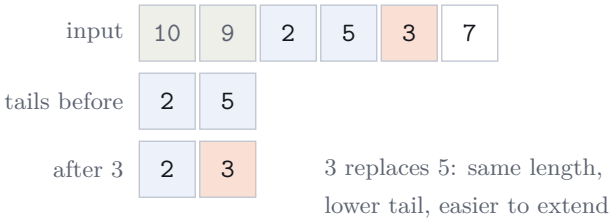
    return best[amount] == Integer.MAX_VALUE ? -1 : best[amount];
}
```

Cost. $O(\mathit{amount} \times \mathit{coins})$ time, $O(\mathit{amount})$ space.

Trap. The greedy "take the biggest coin that fits" is wrong on coin sets like $\{1, 3, 4\}$: making 6 greedily takes $4 + 1 + 1$, while the answer is $3 + 3$. It happens to work on real currency, which is why people trust it.

P60. Longest increasing subsequence — Hard

Length of the longest strictly increasing subsequence; the elements need not be adjacent. `[10,9,2,5,3,7,101,18]` gives 4.
Idea. Keep a list `tails`, where `tails[i]` is the smallest value that can end an increasing subsequence of length $i + 1$. Place each new element by binary search.



How it works. `tails` is always sorted, which is what makes the binary search legal. Each new element does one of two things: if it is larger than everything in the list it extends it by one; otherwise it overwrites the smallest tail that is at least as large.

That overwrite does not lengthen any subsequence, but it lowers the bar for future elements, which is why it is worth doing. The subtle point is that `tails` is *not* a valid subsequence of the input — only its length is meaningful. If the problem asks you to reconstruct the actual subsequence you need to record predecessors as well.

Solution

```java
public static int lengthOfLIS(int[] a) {
    if (a == null || a.length == 0) return 0;
    int[] tails = new int[a.length];
    int size = 0;

    for (int x : a) {
        int lo = 0, hi = size;
        while (lo < hi) {                       // first tail >= x
            int mid = (lo + hi) >>> 1;
            if (tails[mid] < x) lo = mid + 1;
            else hi = mid;
        }
        tails[lo] = x;
        if (lo == size) size++;
    }
    return size;
}
```

Cost. $O(n \log n)$ time, $O(n)$ space. The straightforward $O(n^2)$ DP is the right answer below a few thousand elements and far easier to get right.

Trap. `(lo + hi) / 2` overflows for huge bounds; `>> 1` is the habit worth having. And for a *non-strictly* increasing subsequence the search changes to "first tail > x" — a one-character difference that silently changes the answer.